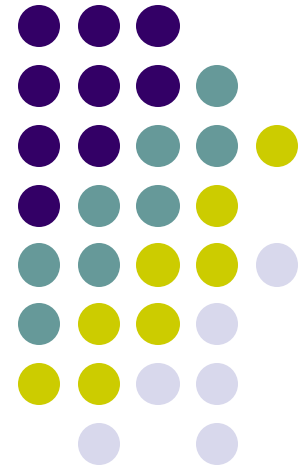
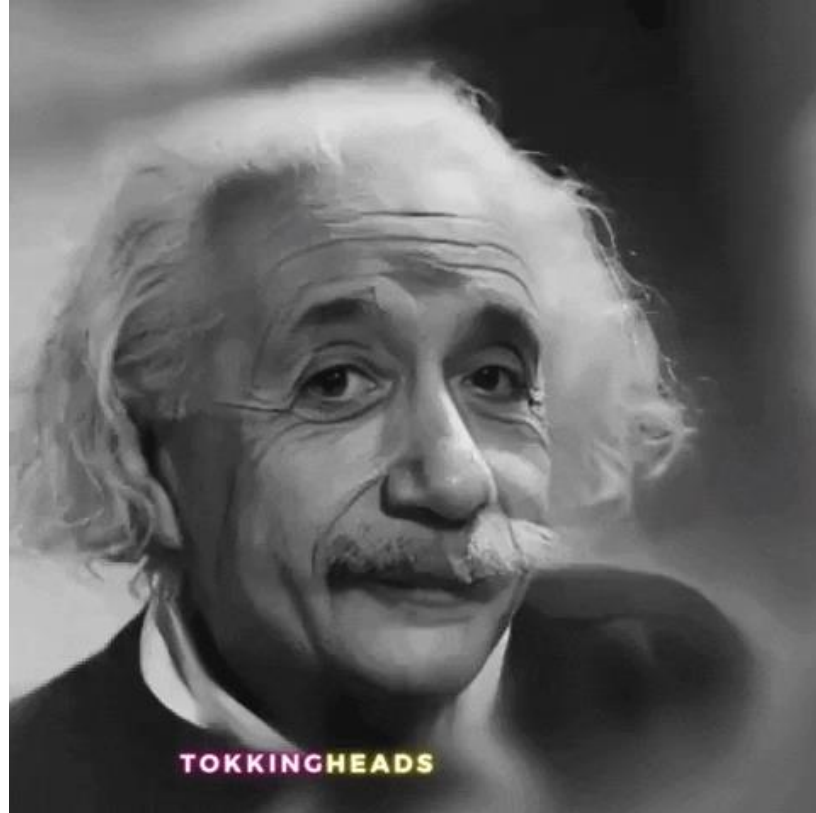
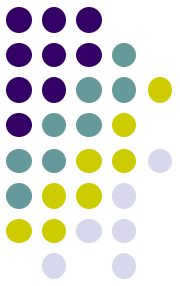


ErSE222: Generative AI: Diffusion models

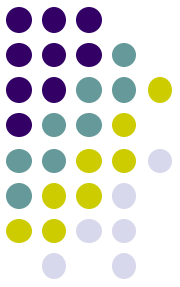
April. 27th, 2025



Generative



ChatGPT: How would seismic waves sound like in poetry?



*A tremor in the earth's deep chest,
A low, vibrating bass.*

*The P-wave pulses, fast and swift,
With every breath, it makes a shift.*

S-waves ripple, slower hum,

A quiver, rising, gently strum.

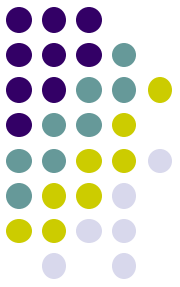
And on the surface, waves of Love

Roll like whispers from above.

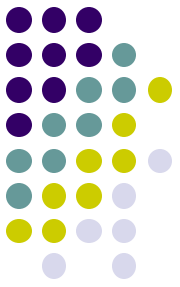
The earth sings out a rumbling song,

Both harsh and soft, short and long.

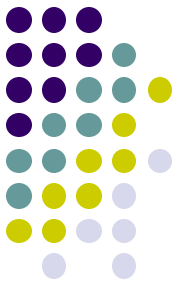
DALE-E 3: An image that depicts KAUST



DALE-E 3



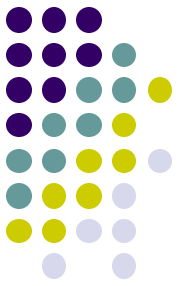
The goal of generative models



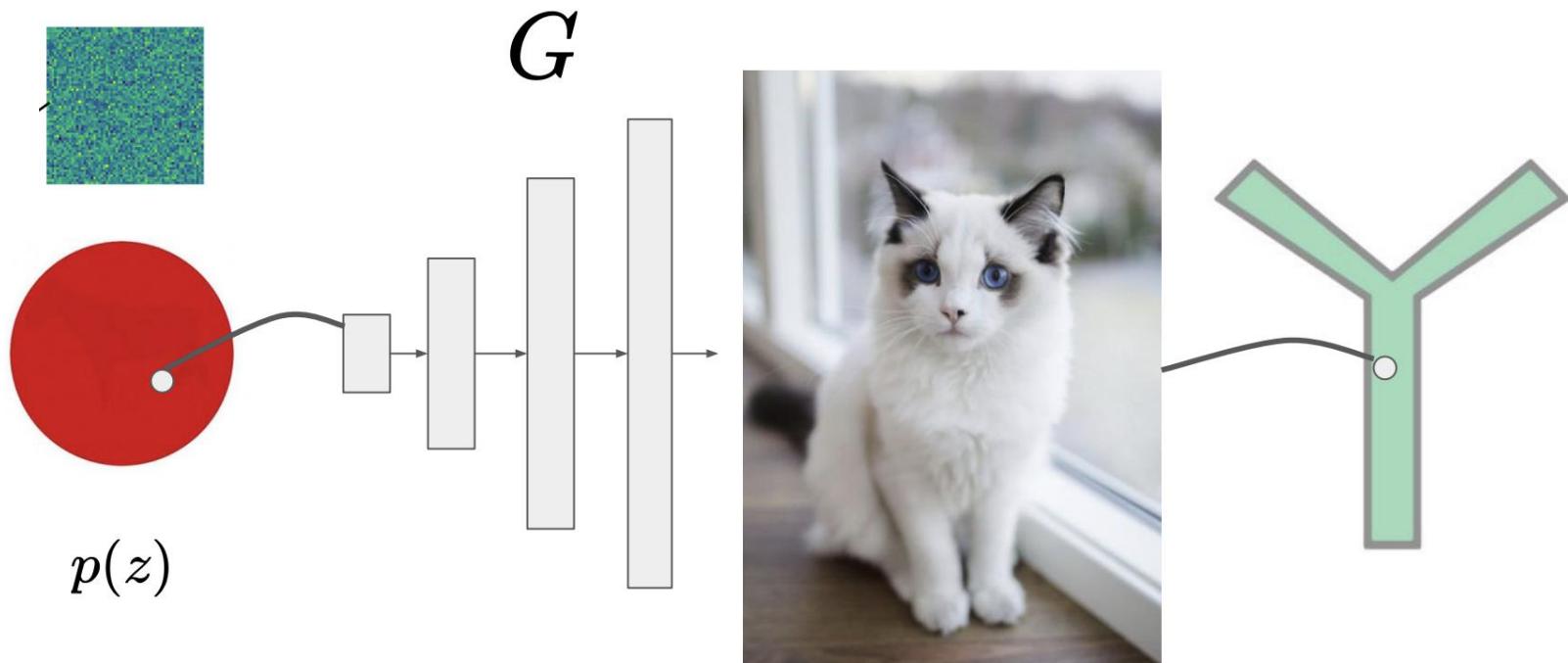
Given a distribution $p(x)$ represented by data samples

- Learn the underlying distribution $p_{\theta}(x)$ from its sample representation (we do not have knowledge of complex distributions)
- Sample $x \sim p_{\theta}(x)$ (new unseen samples) from the learned distribution
- We train to maximize the log likelihood of $p_{\theta}(x)$, or minimize divergence between $p_{\theta}(x)$ and $p_{data}(x)$

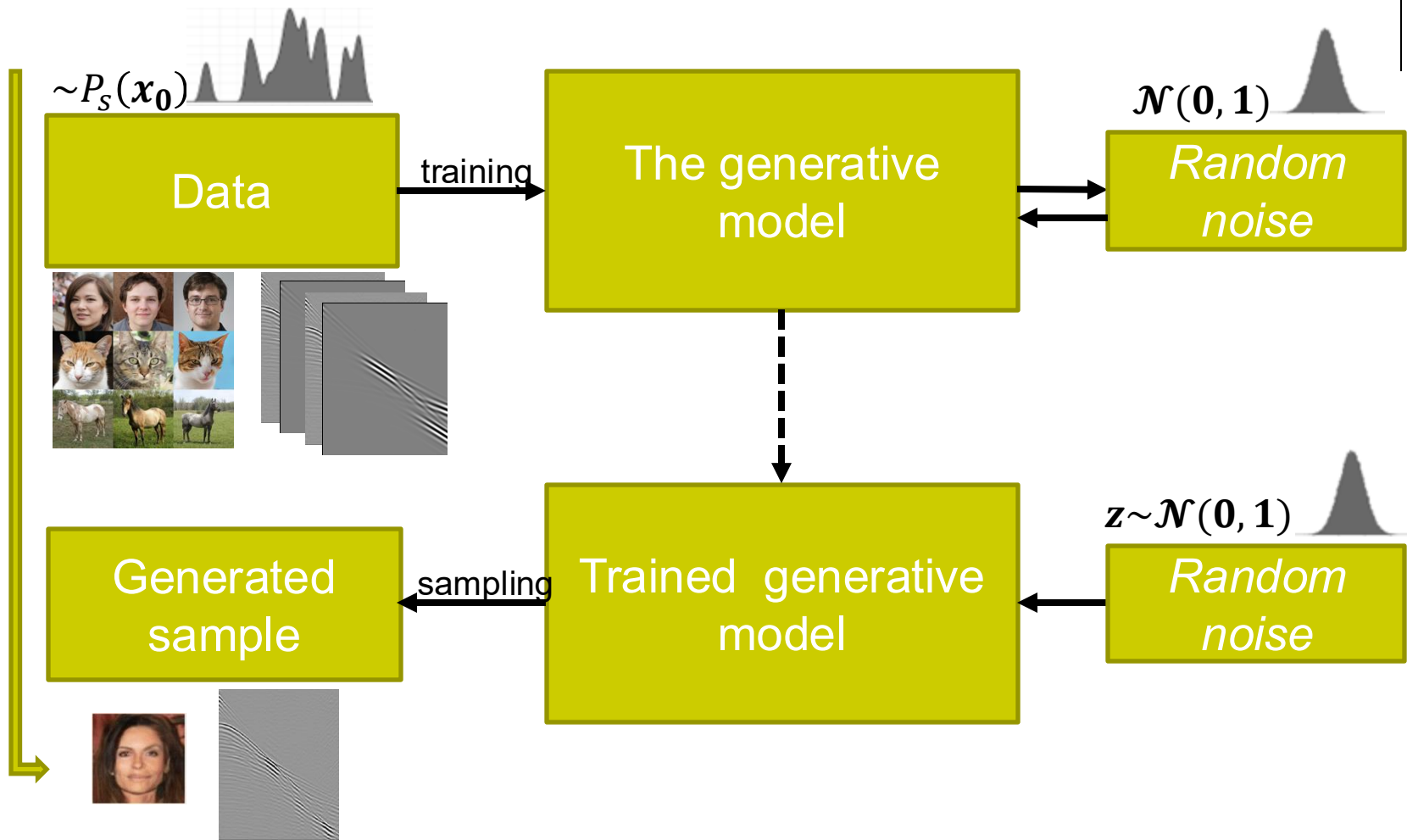
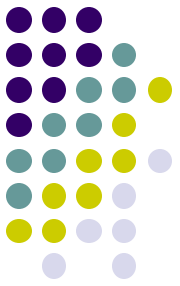
A neural network for Distributions



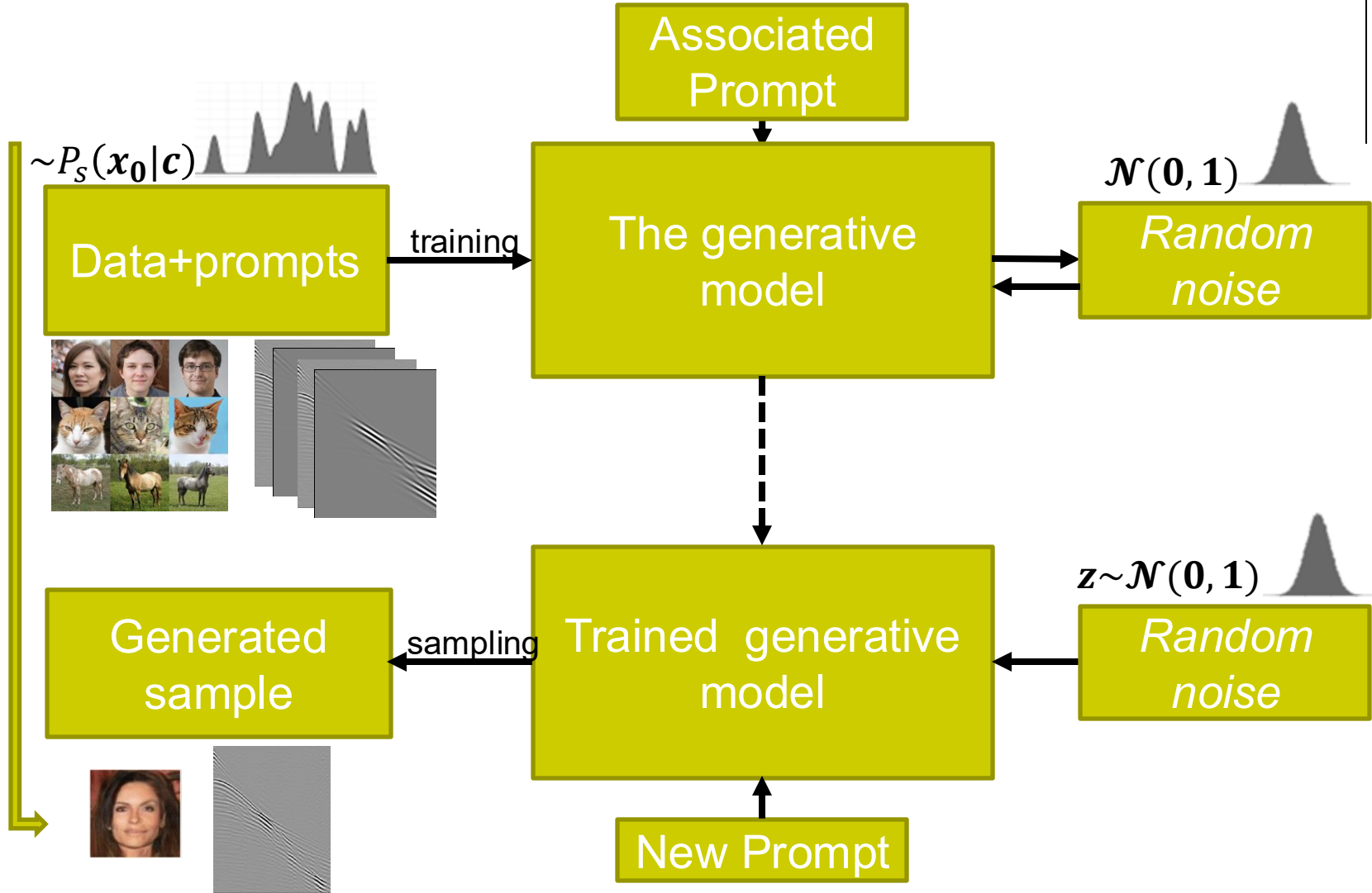
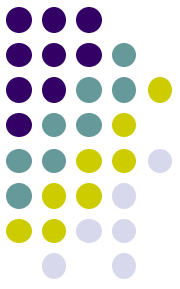
This is done by mapping the complex distribution to a simple distribution we can represent analytically



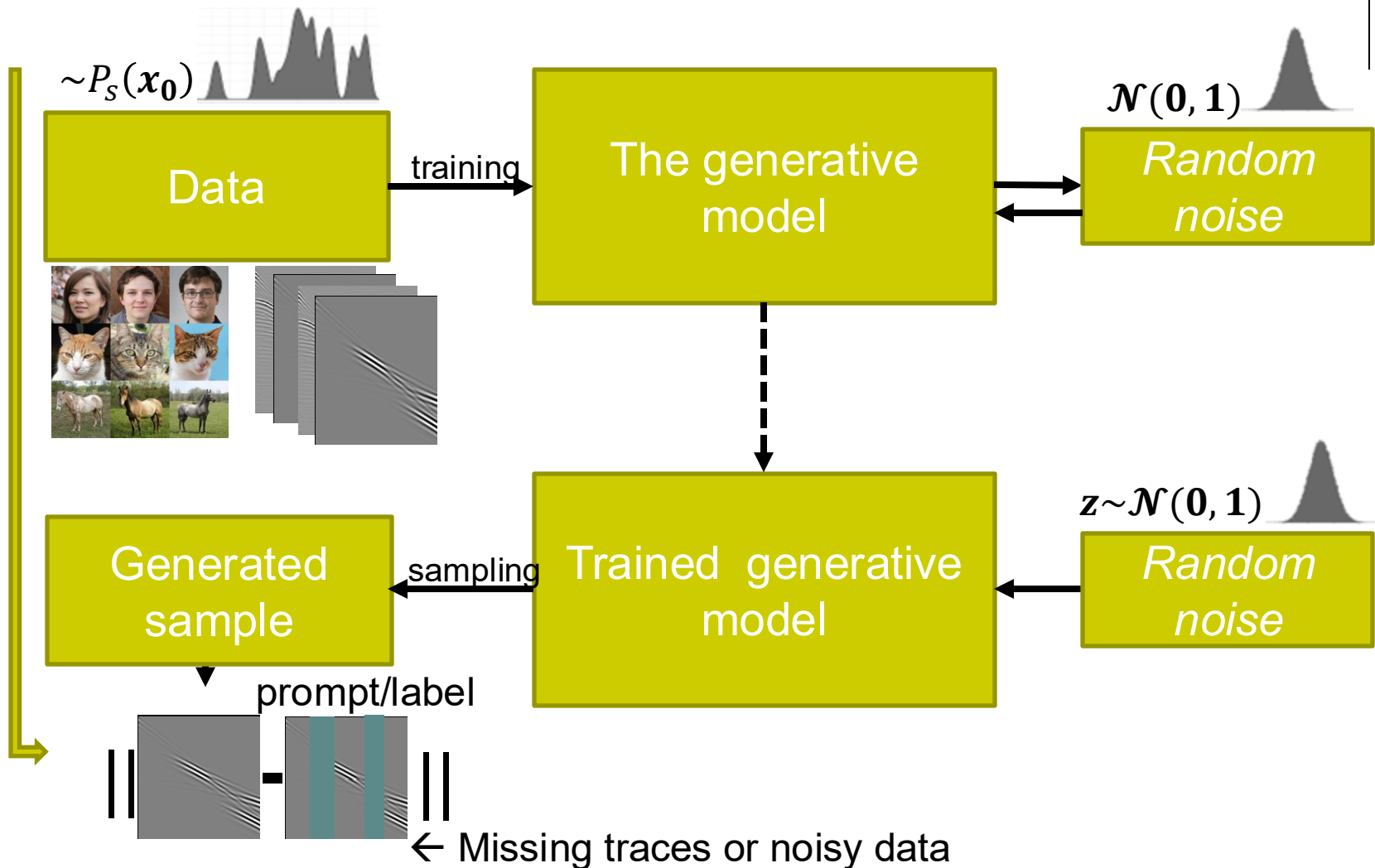
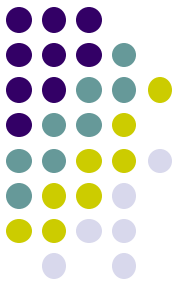
The generative model: sampling from complex distributions



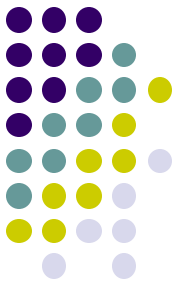
Conditional



Guidance



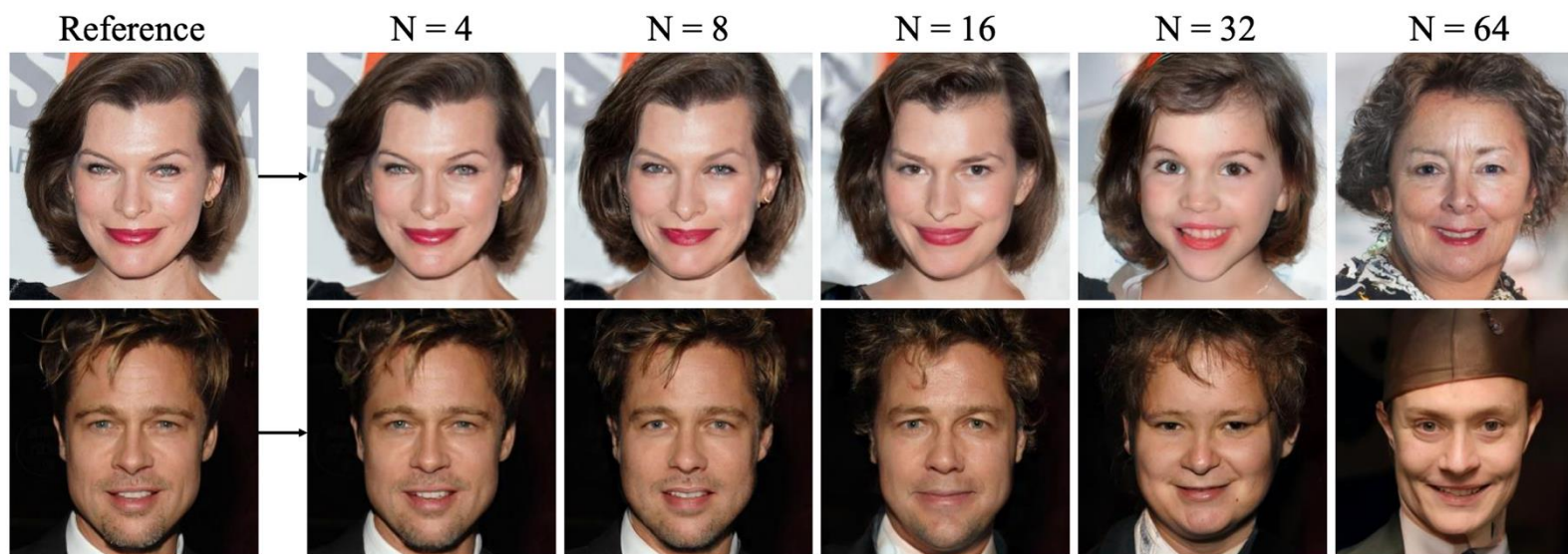
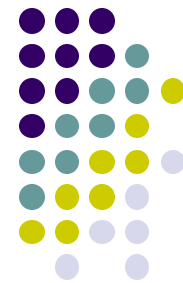
Bayesian Theorem



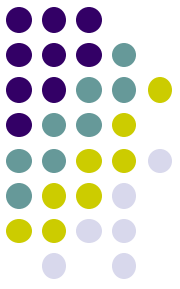
- The data as condition or guidance.

$$P(x|y) = \frac{P(y|x)P(x)}{P(y)}$$

With Guidance --- The inverse problem

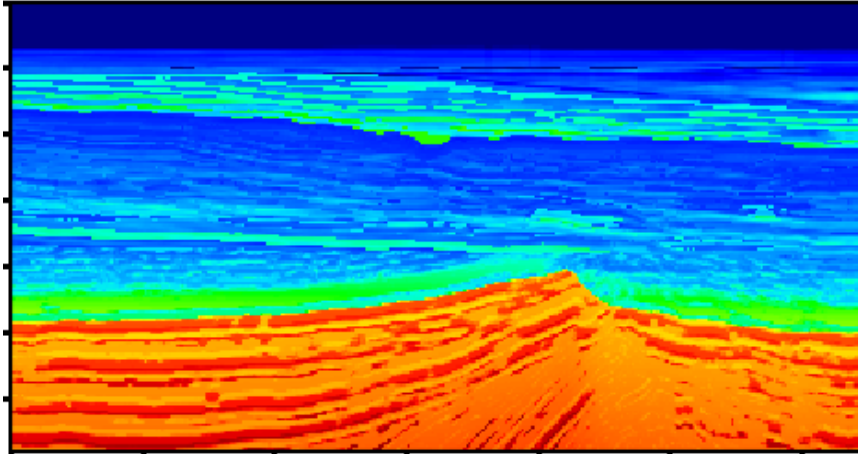


¹Choi et al. (2021)

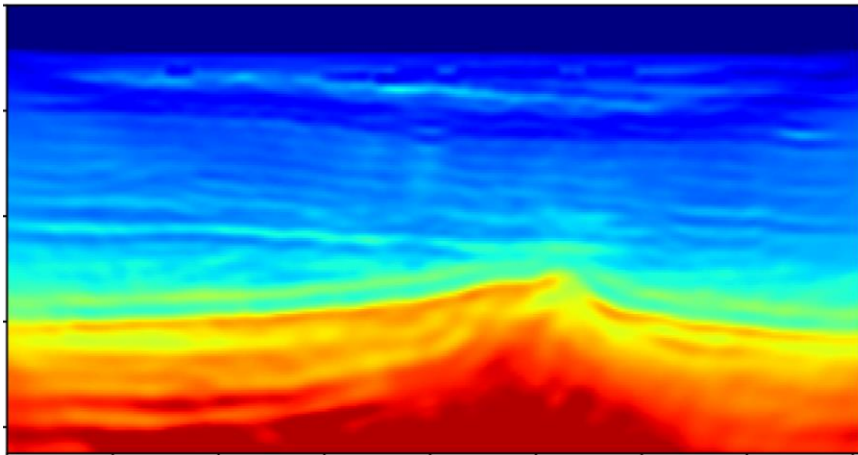


Inverse problems: storing expectations (potential solutions)

Geologically based BG Compass model



FWI result

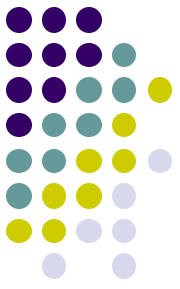


- Faults
- Fractures
- Spatial distribution (increase with)
- Layering
- Reservoir
-

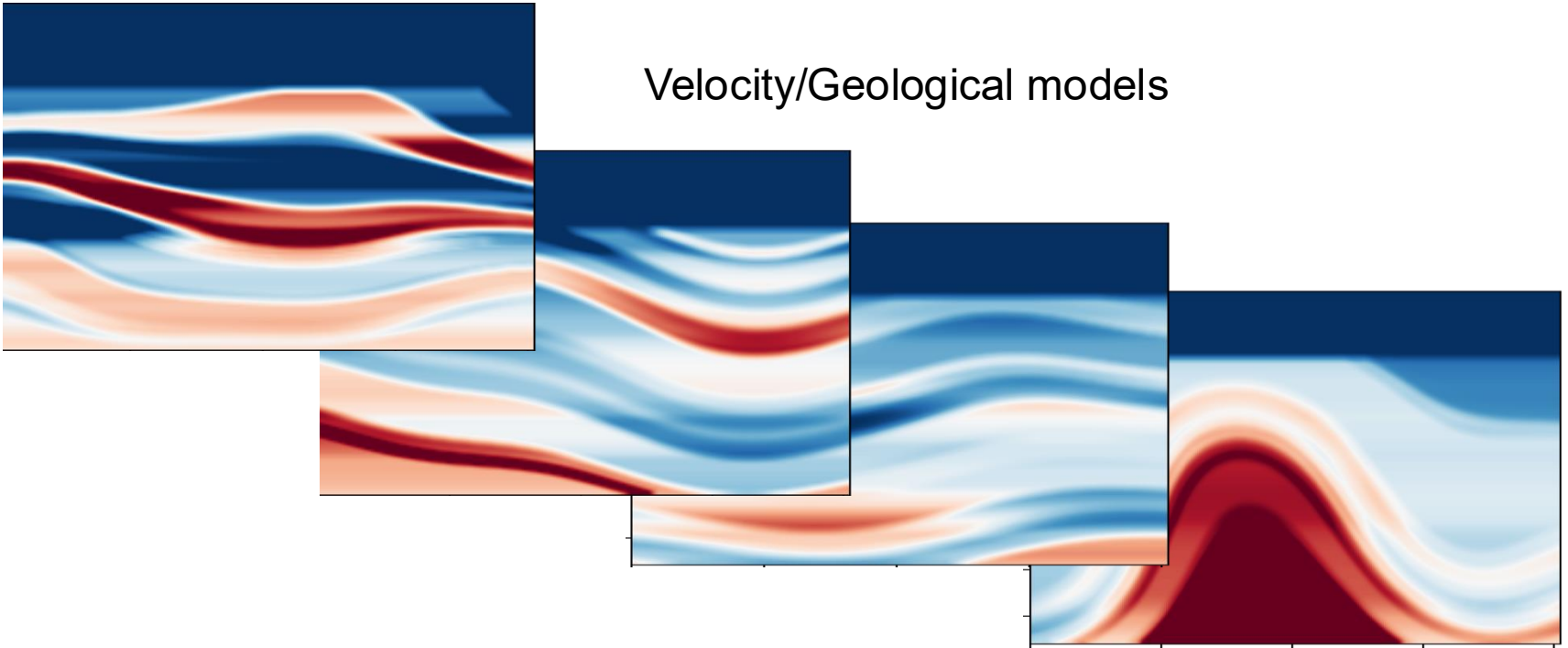


The generative opportunity:

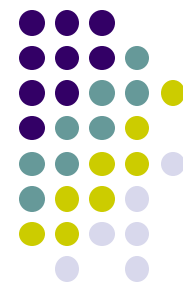
storing a (prior) distribution that covers the potential solution from the solution space in which the network learns the structure of our solution



Velocity/Geological models

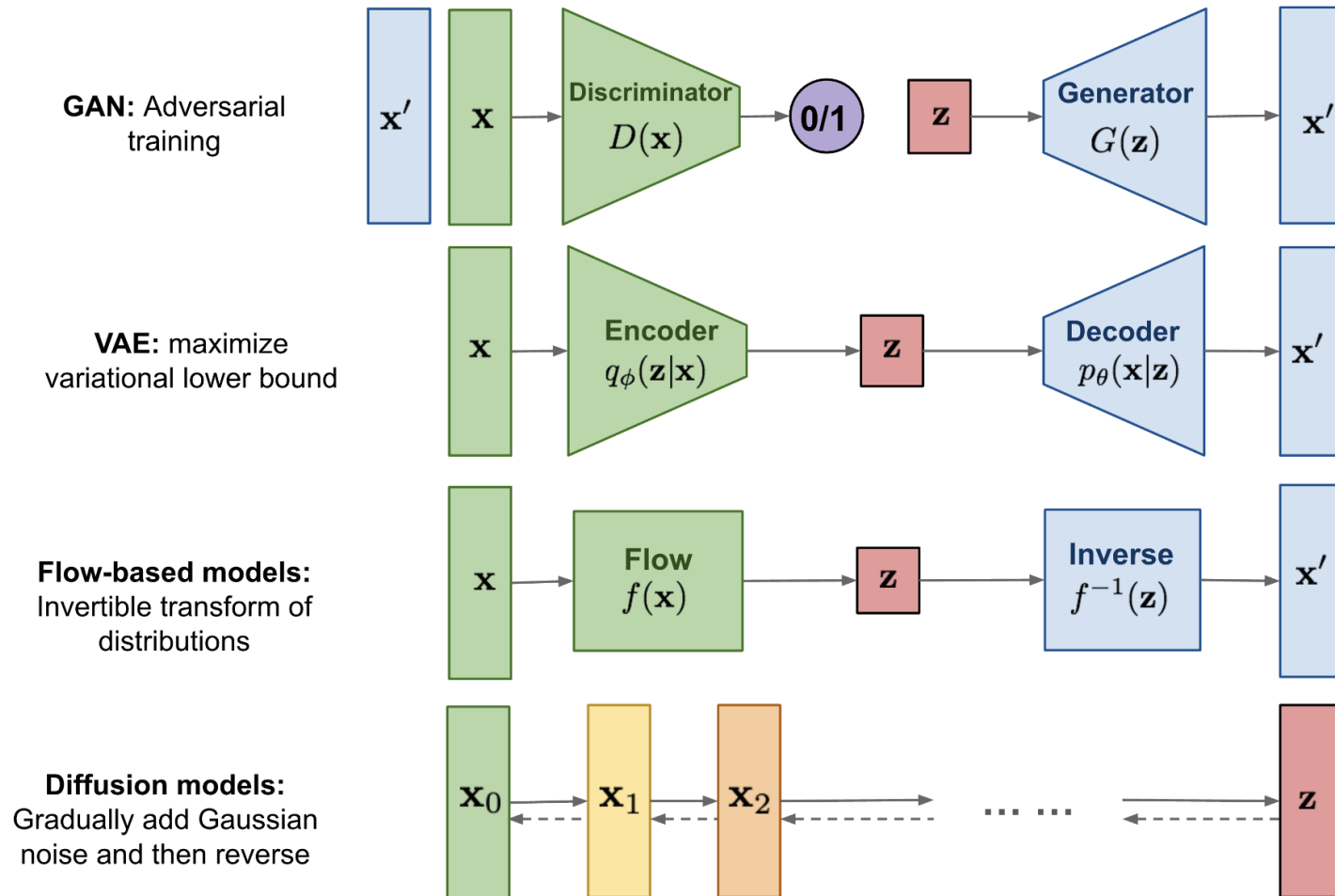
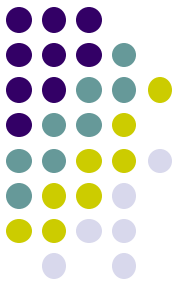


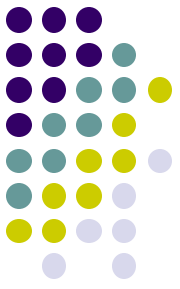
Distributions we store?



- Velocity/Elastic CO2 saturation models (our big objective)
- Salt bodies for flooding and unflooding
- Seismic data (for processing)
- Seismic wavefields
- Location of microseismic events (regression or segmentation)

The generative options:

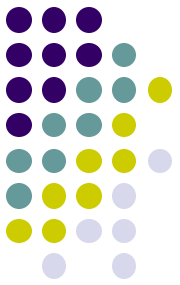




What are Diffusion Models?

- A class of generative models using stochastic processes
- Learn data distribution by reversing a noise process
- Applications: Image, Audio, and Video Synthesis

Prior Methods



VAE:

$$\begin{aligned} L_{\text{VAE}}(\theta, \phi) &= -\log p_{\theta}(\mathbf{x}) + D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p_{\theta}(\mathbf{z}|\mathbf{x})) \\ &= -\mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} \log p_{\theta}(\mathbf{x}|\mathbf{z}) + D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p_{\theta}(\mathbf{z})) \\ \theta^*, \phi^* &= \arg \min_{\theta, \phi} L_{\text{VAE}} \end{aligned}$$

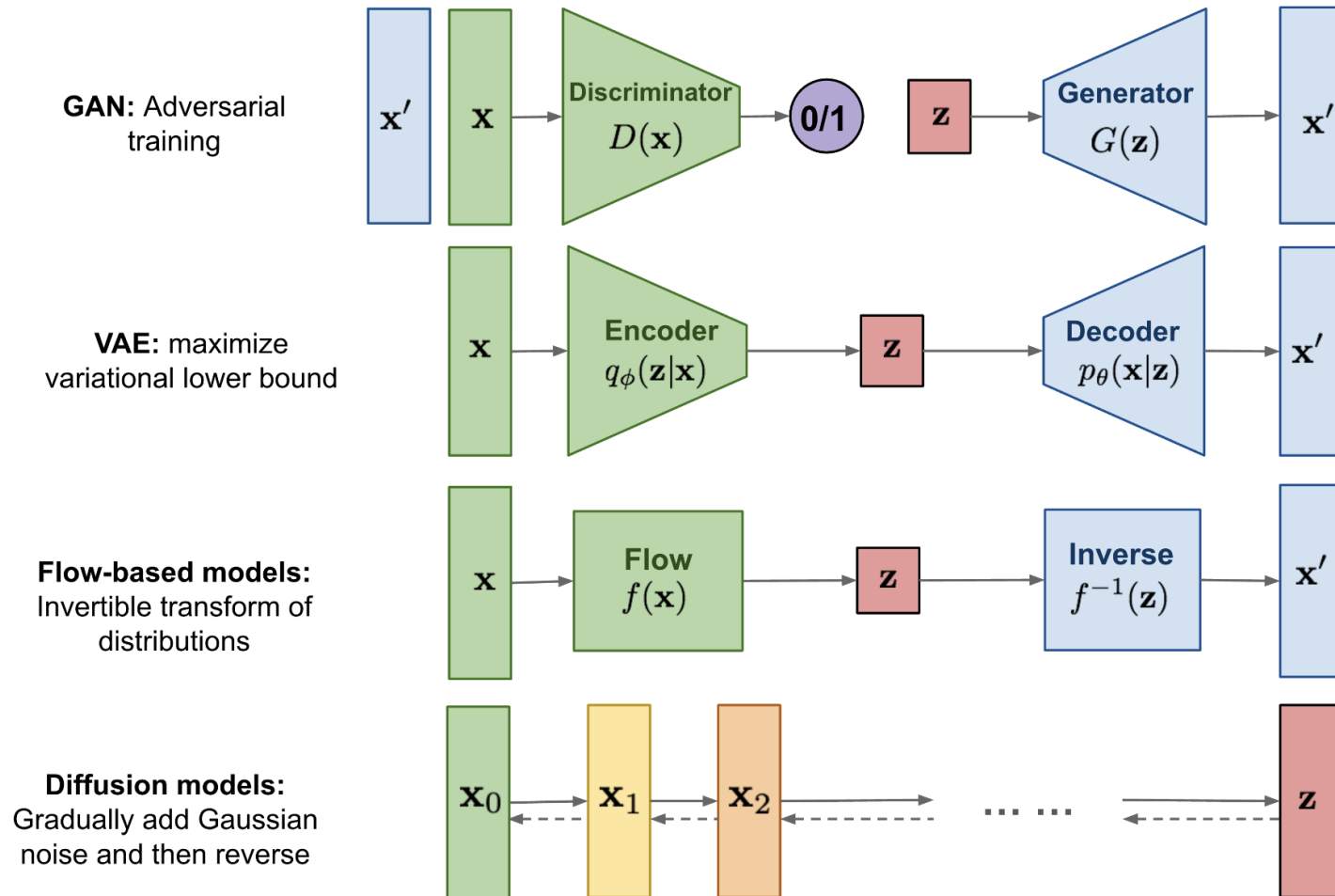
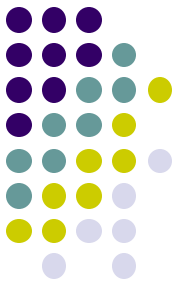
$$-L_{\text{VAE}} = \log p_{\theta}(\mathbf{x}) - D_{\text{KL}}(q_{\phi}(\mathbf{z}|\mathbf{x}) \| p_{\theta}(\mathbf{z}|\mathbf{x})) \leq \log p_{\theta}(\mathbf{x})$$

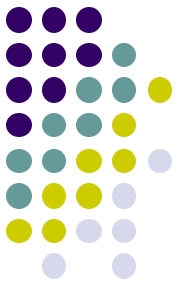
GAN:

$$\begin{aligned} \min_G \max_D L(D, G) &= \mathbb{E}_{x \sim p_r(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \\ &= \mathbb{E}_{x \sim p_r(x)} [\log D(x)] + \mathbb{E}_{x \sim p_g(x)} [\log(1 - D(x))] \end{aligned}$$

$$L(G, D^*) = 2D_{JS}(p_r \| p_g) - 2 \log 2$$

Sampling from noise:

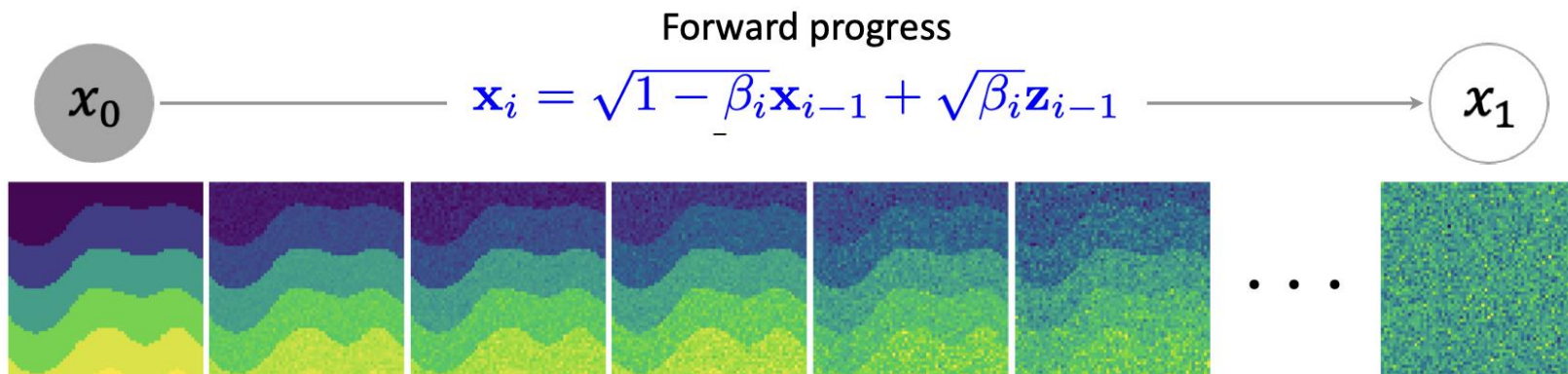




The Diffusion framework

- Another kind of generative models, inspired by physics, based on SDE.
- Convert a well known distribution to the desired learned one through small steps, using the Markov chain
 - Forward Process
 - Reverse Process

The Forward Process (Diffusion)



- **Gradually add Gaussian noise** to data over T steps:

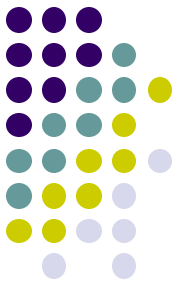
$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t \mathbf{I})$$

- x_t : Noisy sample at step t .
- β_t : Noise schedule (small positive constants).

- **Closed-form after T steps:**

$$q(x_T | x_0) \approx \mathcal{N}(x_T; 0, \mathbf{I})$$

The Full Forward Process



From data $\mathbf{x}_0$ to noisy image $\mathbf{x}_T$

$$q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

Sampling at any step:

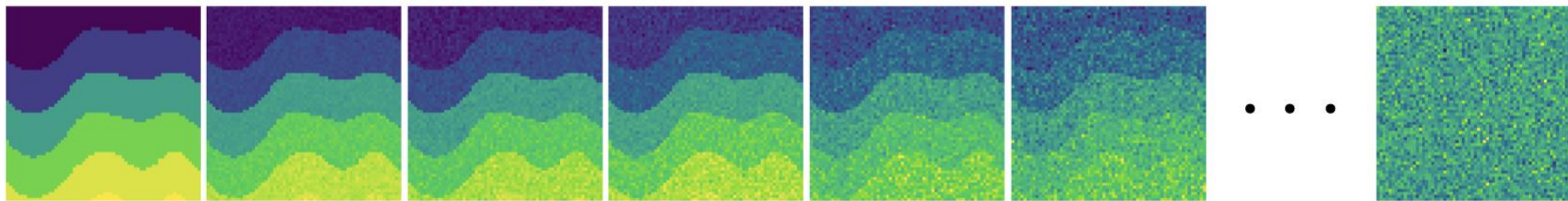
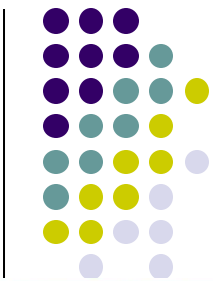
$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})$$

where $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$

In the limit, $\mathbf{x}_\infty$ is a random vector from an isotropic Gaussian

$$\lim_{t \rightarrow \infty} \mathbf{x}_t = \sqrt{\bar{\alpha}_\infty} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_\infty} \boldsymbol{\epsilon} = \boldsymbol{\epsilon} \text{ since } \bar{\alpha}_\infty \rightarrow 0$$

The Reverse Process



$$\mathbf{x}_0 \longleftarrow dx = \left[-\frac{\beta(t)}{2} \mathbf{x} - \beta(t) \nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t) \right] dt + \sqrt{\beta(t)} d\mathbf{w} \longrightarrow \mathbf{x}_1$$

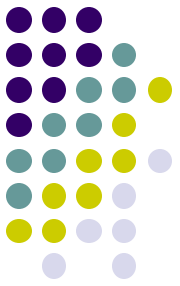
Reverse progress

- **Goal:** Learn to reverse $q(\mathbf{x}_t | \mathbf{x}_{t-1})$ by estimating:

$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_{\theta}(\mathbf{x}_t, t), \Sigma_{\theta}(\mathbf{x}_t, t))$$

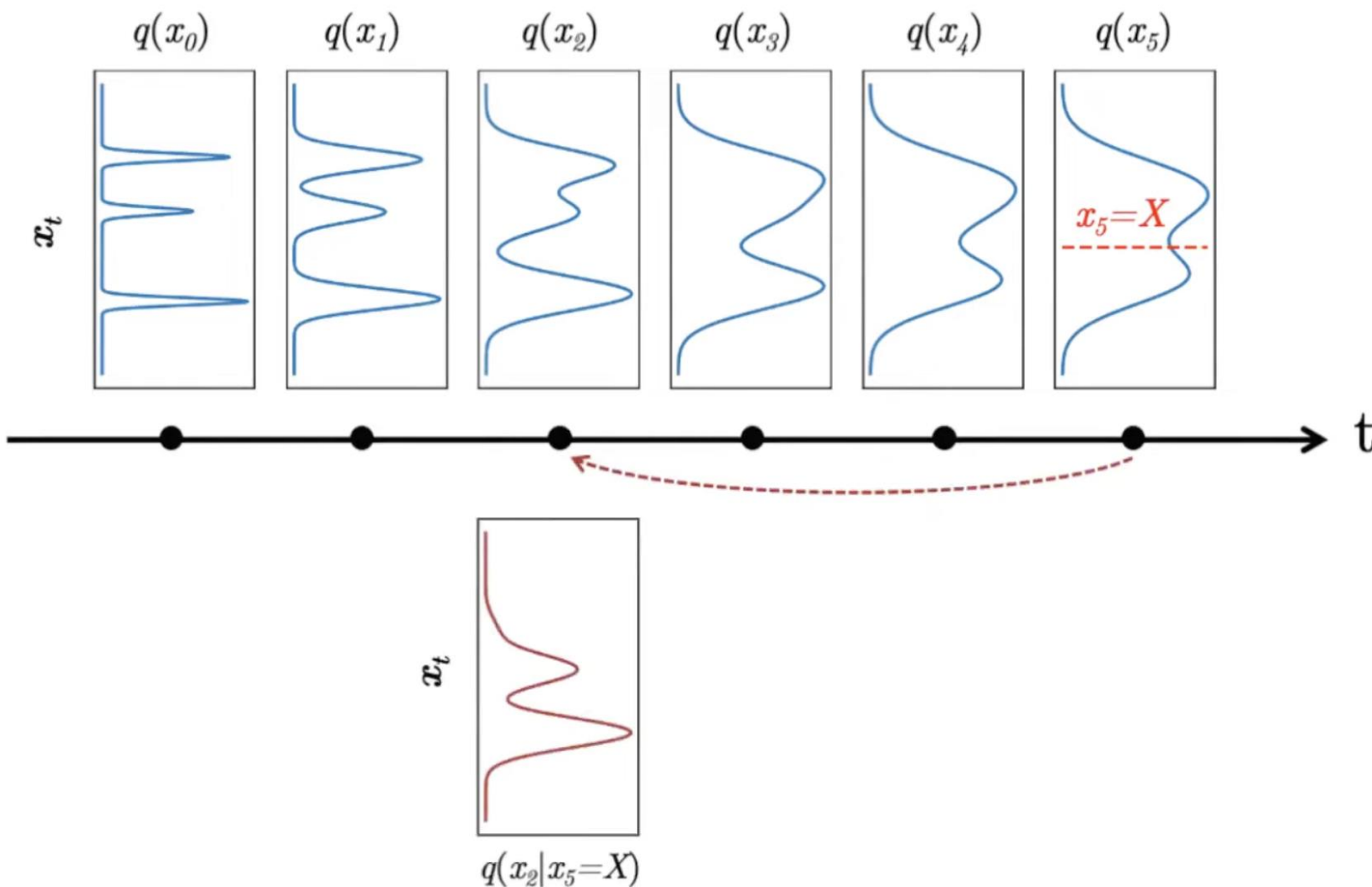
- θ : Neural network parameters.
- **Key Challenge:** Estimating noise ϵ at each step.

The Chain

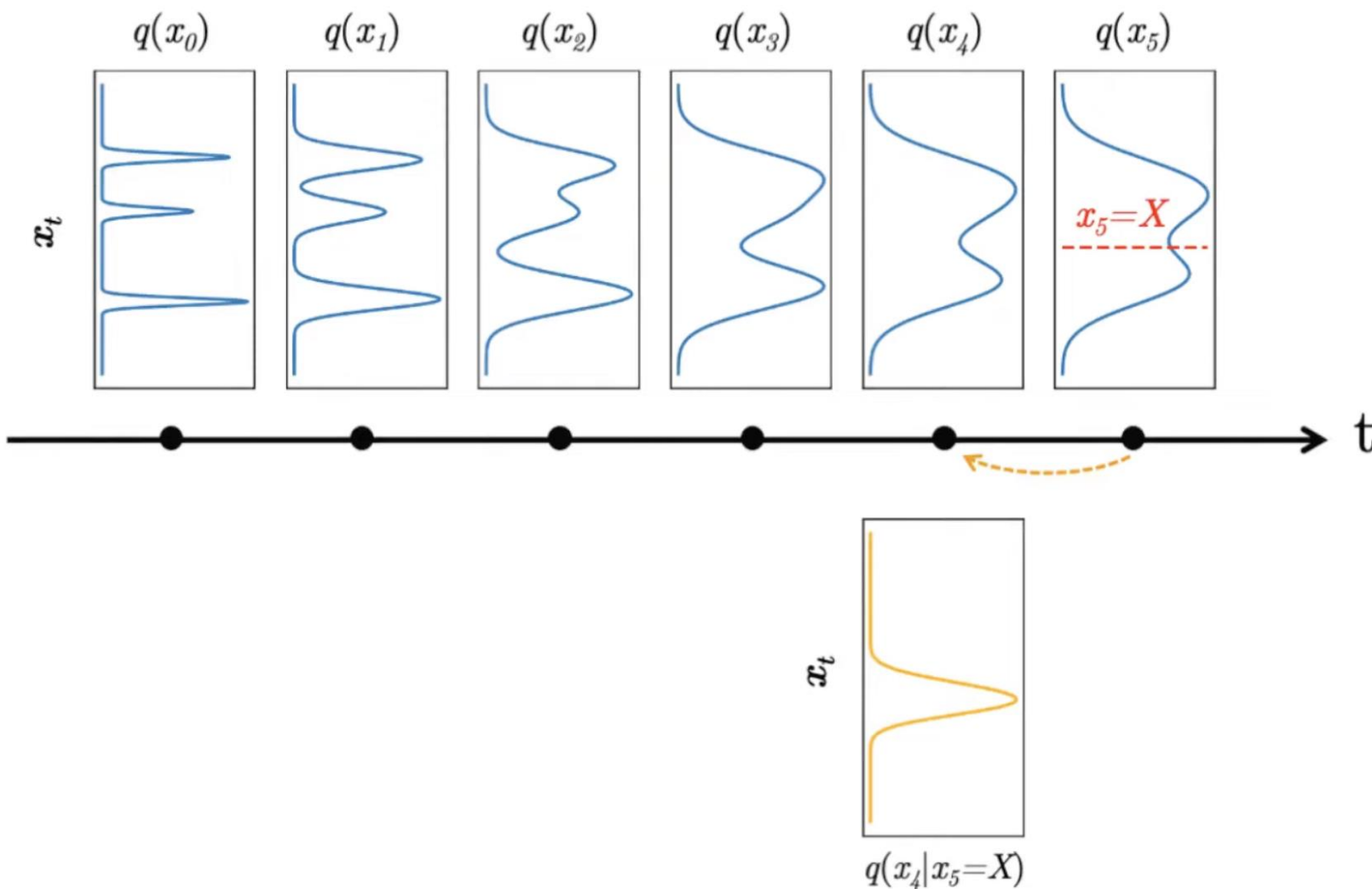


- Forward factorization: $q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) = q(\mathbf{x}_0) \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1})$
- Reverse factorization: $q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) = \prod_{t=1}^T q(\mathbf{x}_{t-1} | \mathbf{x}_t) q(\mathbf{x}_T)$
 - Since joint distribution is Gaussian then $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ is also Gaussian
 - $q(\mathbf{x}_{t-1} | \mathbf{x}_t) = N(\mathbf{x}_{t-1} | \tilde{\mu}_t(\mathbf{x}_t, t), \sigma_t I)$

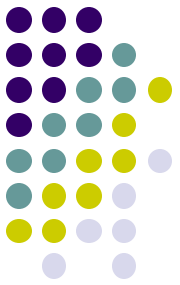
One dimension



Normal



The neural approximation



- The reverse conditional $q(\mathbf{x}_{t-1}|\mathbf{x}_t) = N(\mathbf{x}_{t-1}|\tilde{\mu}_t(\mathbf{x}_t, t), \sigma_t \mathbf{I})$ does not have a closed form, but Ho, Jain and Abbeel (2020) derived the following approximation for $\tilde{\mu}_t$:

$$\tilde{\mu}_t(\mathbf{x}_t, t) \approx \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_t \right)$$

where $\boldsymbol{\epsilon}_t = N(\boldsymbol{\epsilon}|\mathbf{0}, \mathbf{I})$ is the noise introduced at step t

- We do not know $\boldsymbol{\epsilon}_t$, but we can train a neural network $\epsilon_\theta(\mathbf{x}_t, t)$ to approximate it:

$$\text{Minimize } L(\theta) = \|\boldsymbol{\epsilon}_t - \epsilon_\theta(\mathbf{x}_t, t)\|_2^2$$

The training loss



- **Minimize the variational bound:**

$$\mathcal{L} = \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$$

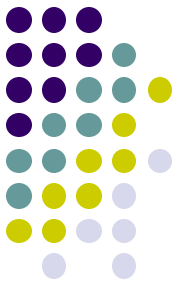
- ϵ : Actual noise added.
- ϵ_θ : Predicted noise by the model.
- **Simplified loss:** Predict noise instead of mean μ_θ .

The loss function breaks into simpler components.

Denoising Score Matching:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} [\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2]$$

The sampling

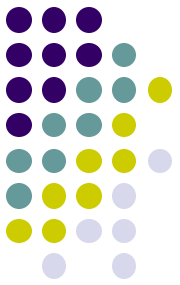


- **Start from pure noise** $x_T \sim \mathcal{N}(0, \mathbf{I})$.
- **Iteratively denoise using the learned model:**

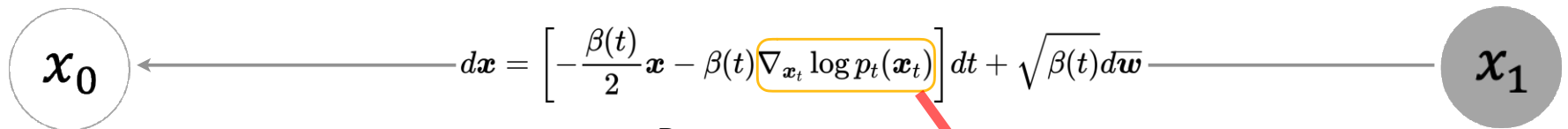
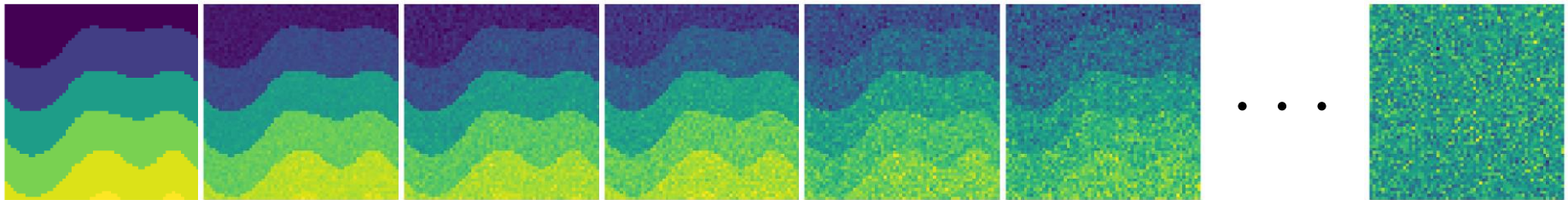
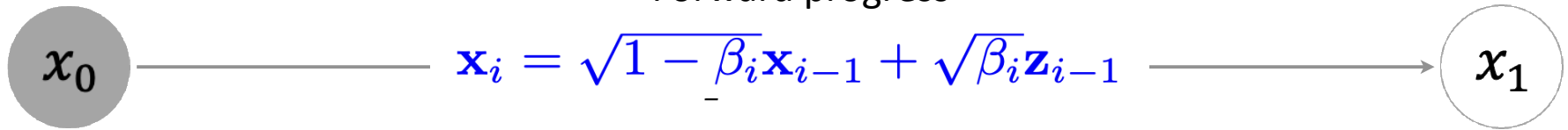
$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z$$

- $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$.
- $z \sim \mathcal{N}(0, \mathbf{I})$ (optional stochastic term).

The Diffusion process



Forward progress

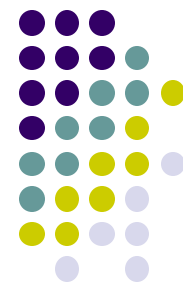


Reverse progress

A red arrow points from the term $\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t)$ in the reverse process equation to the following approximation:

$$\nabla_{\mathbf{x}_t} \log p_t(\mathbf{x}_t) \simeq \mathbf{s}_\theta(\mathbf{x}_t, t)$$

The training Algorithm

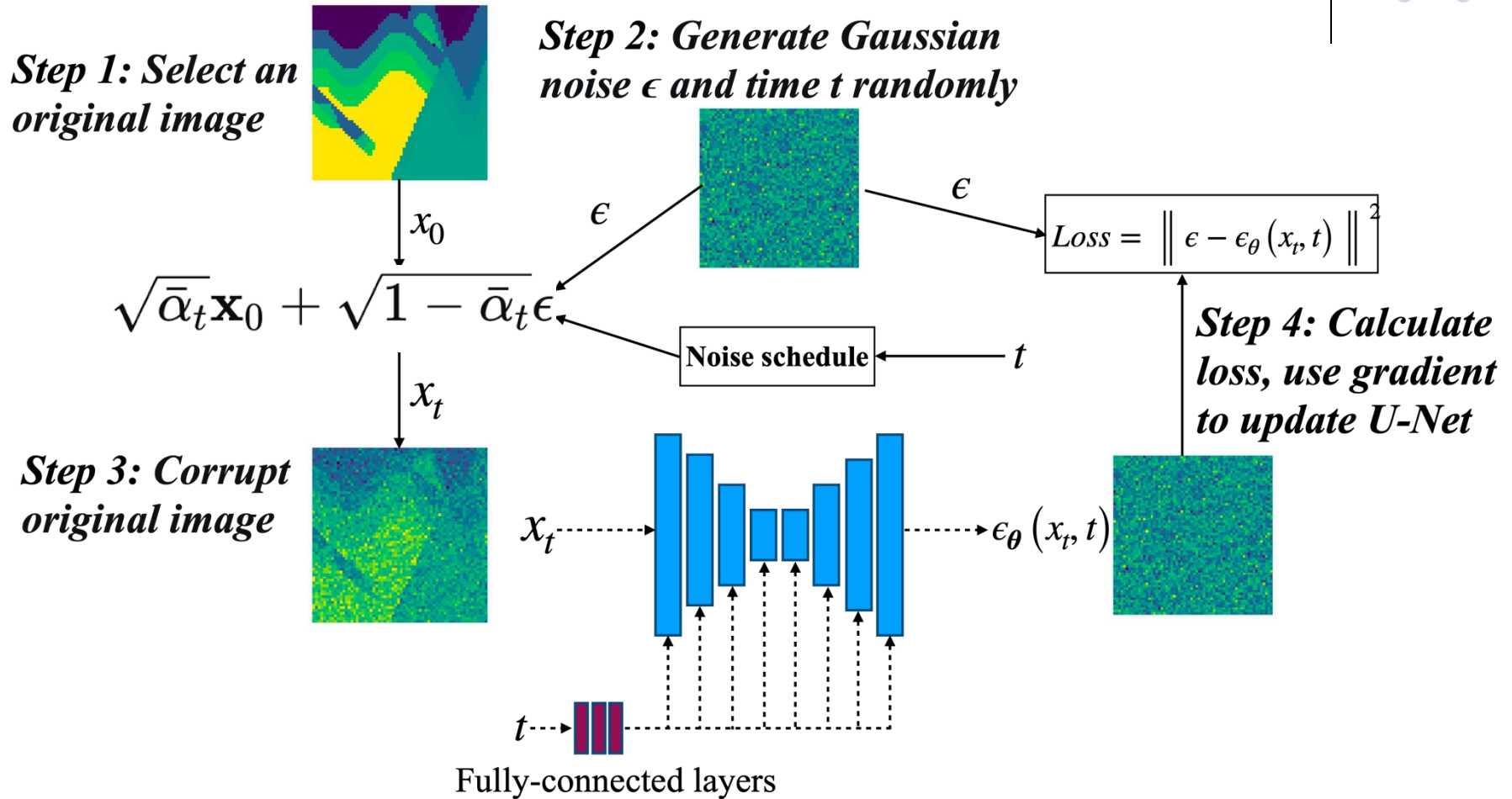
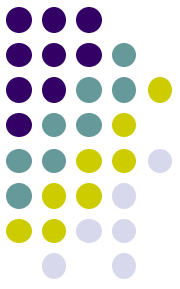


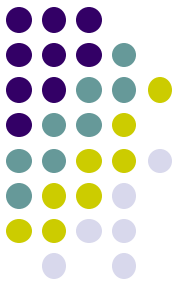
Repeat

- $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
- $t \sim \text{uniform}(\{1, \dots, T\})$
- $\epsilon \sim N(\mathbf{0}, I)$
- $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$
- $\theta \leftarrow \theta + \eta \nabla_{\theta} ||\epsilon - \epsilon_{\theta}(\mathbf{x}_t, t)||_2^2$

Until convergence

The Diffusion training

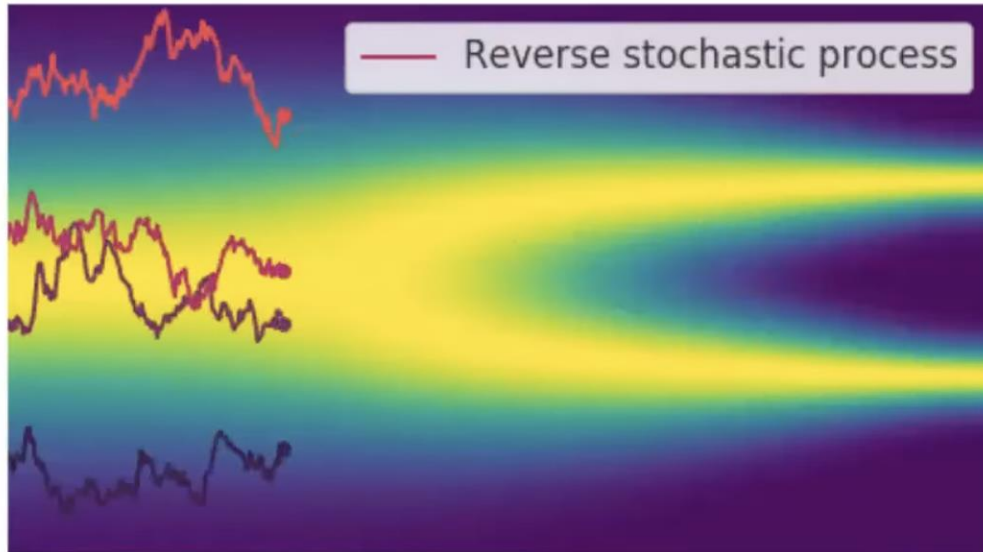
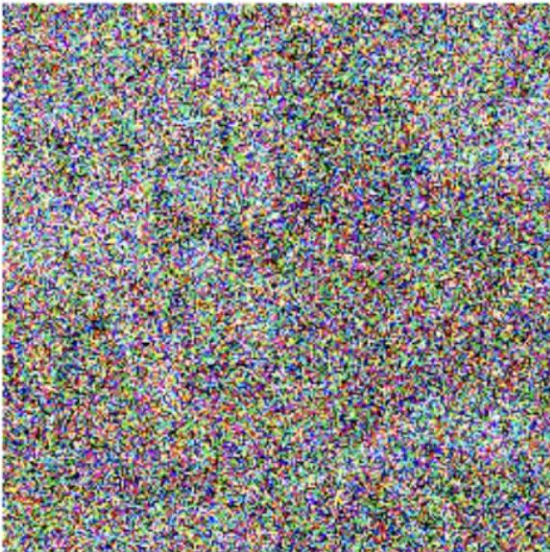
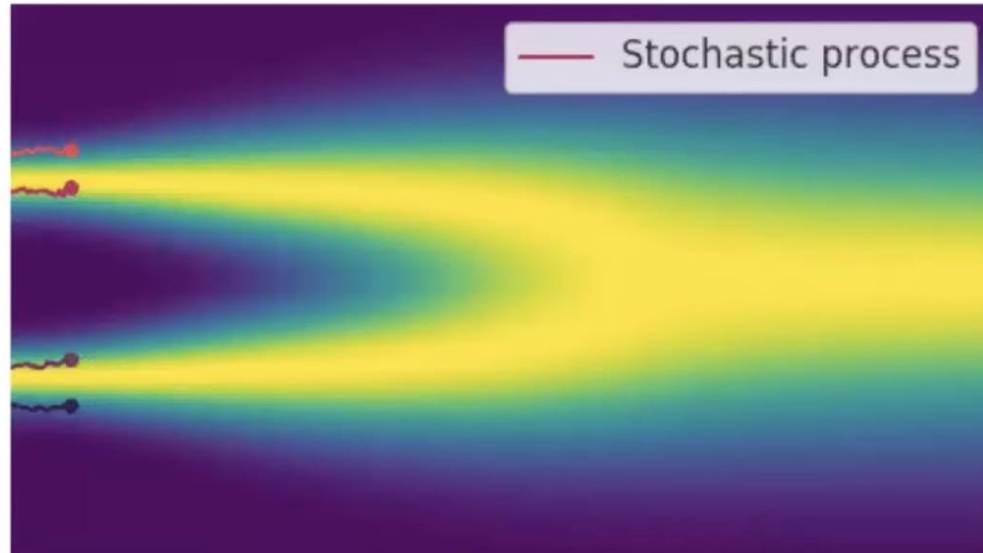
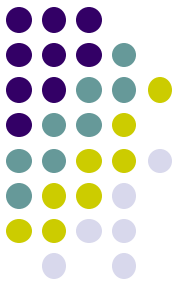




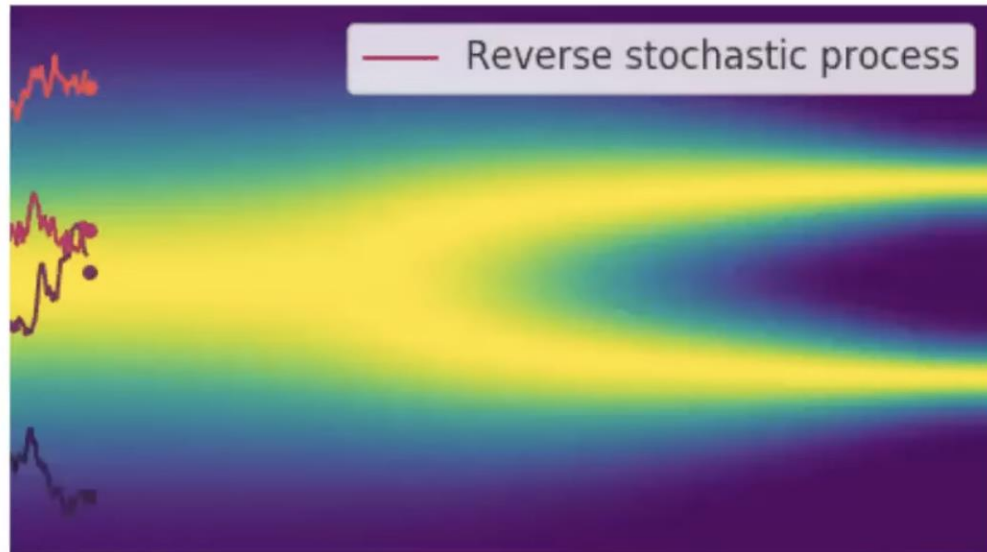
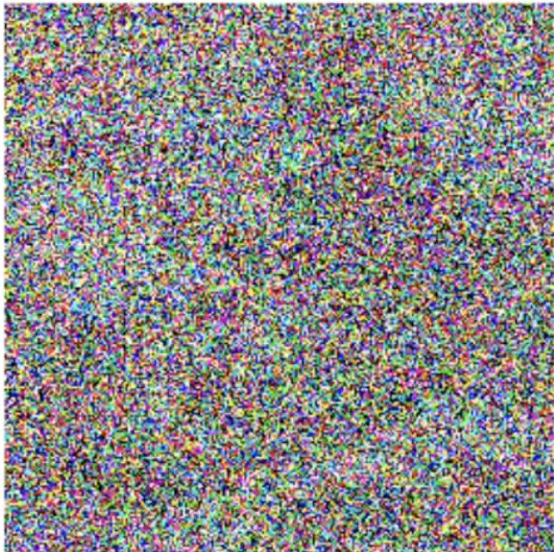
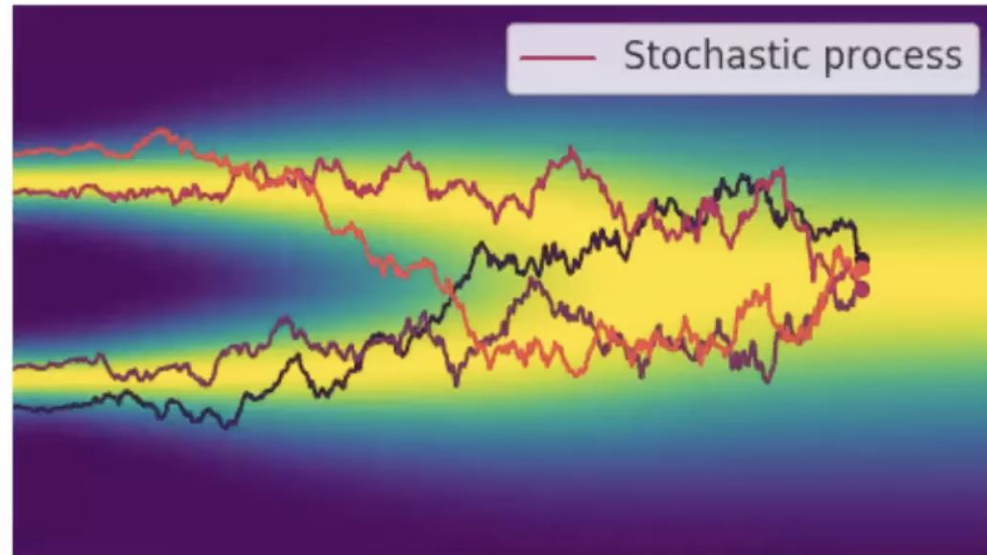
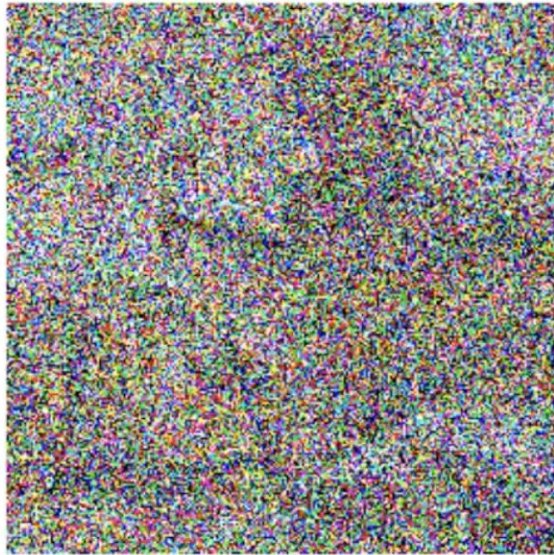
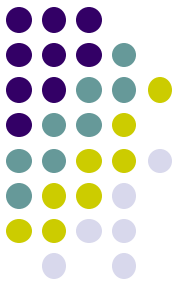
The inference algorithm

- $\mathbf{x}_T \sim N(\mathbf{x}|\mathbf{0}, I)$
- For $t = T, \dots, 1$ do
 - $\epsilon \sim N(\epsilon|\mathbf{0}, I)$ if $t > 1$, else $\epsilon = \mathbf{0}$
 - $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sqrt{\sigma_t} \epsilon$
- Return $\mathbf{x}_0$

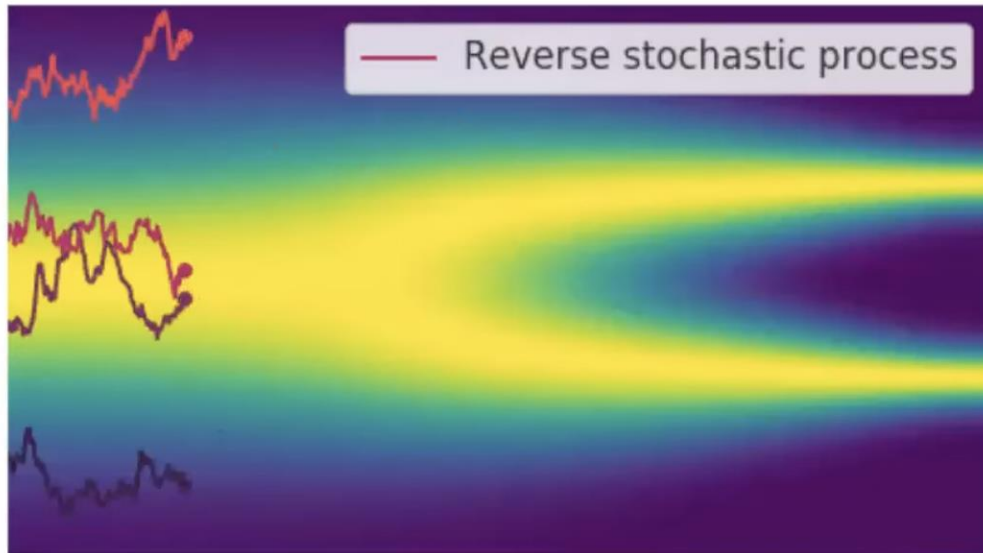
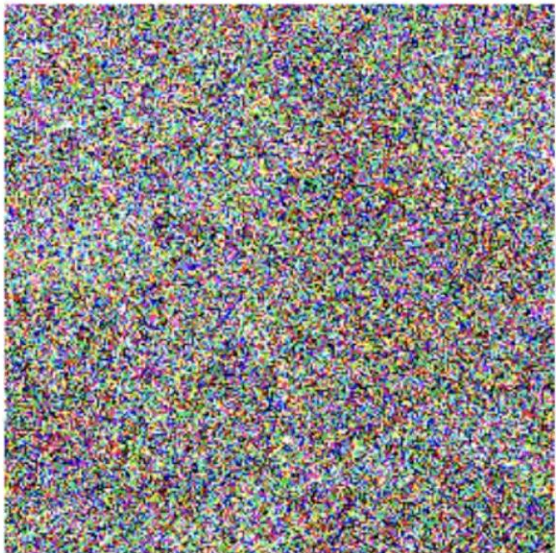
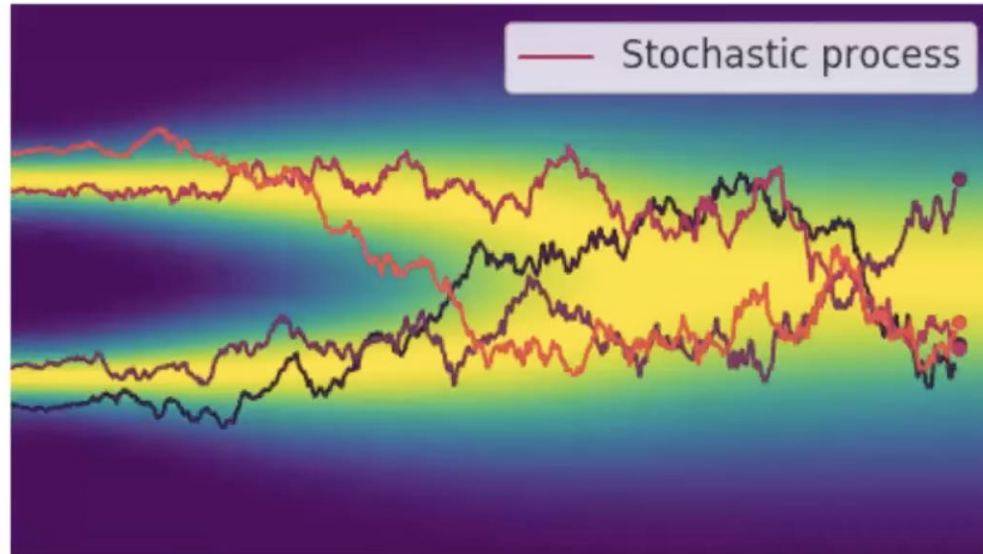
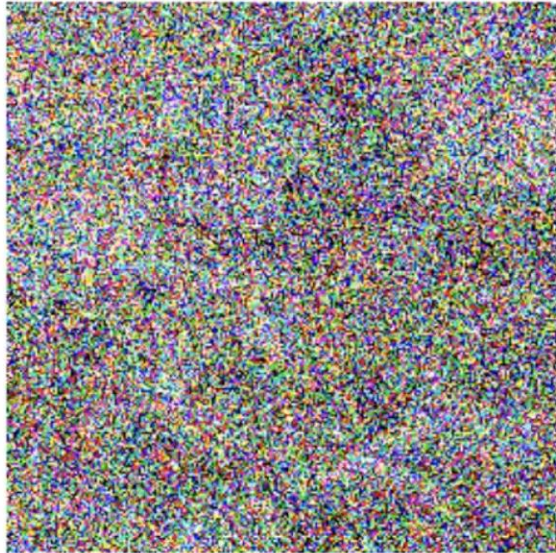
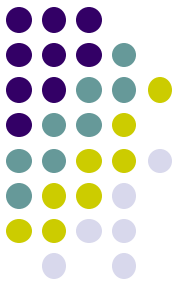
Mapping distributions



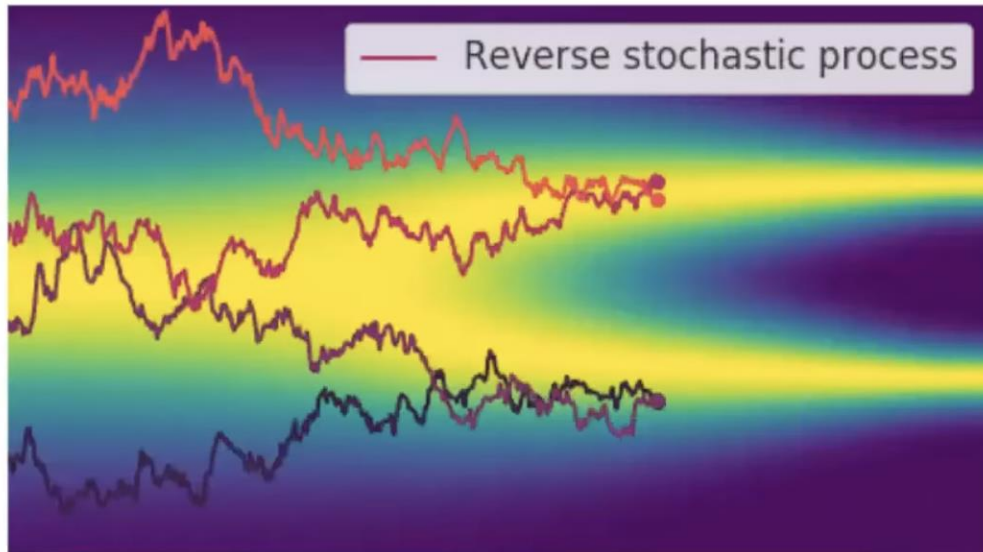
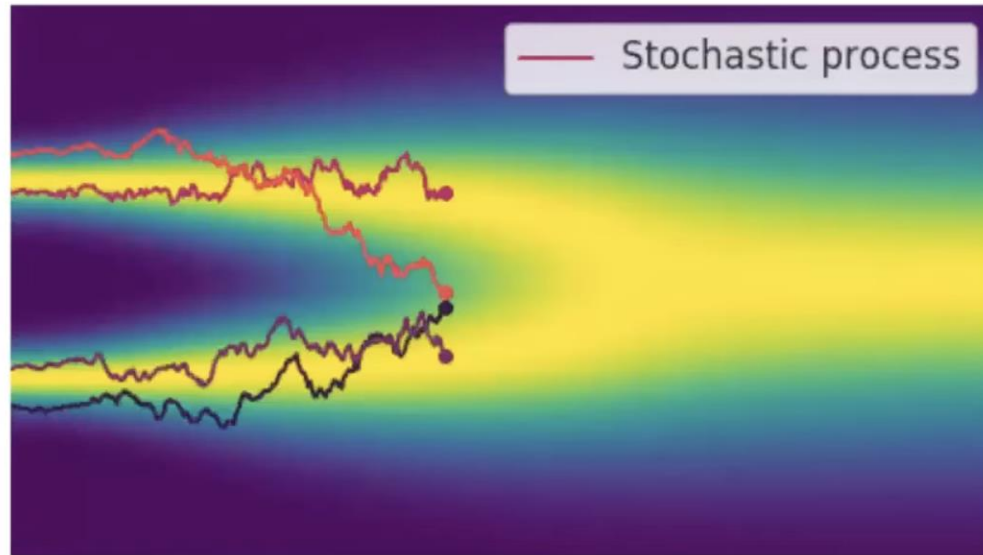
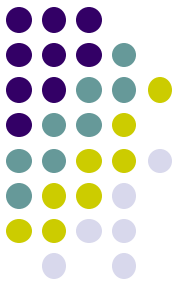
Mapping distributions



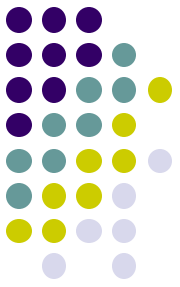
Mapping distributions



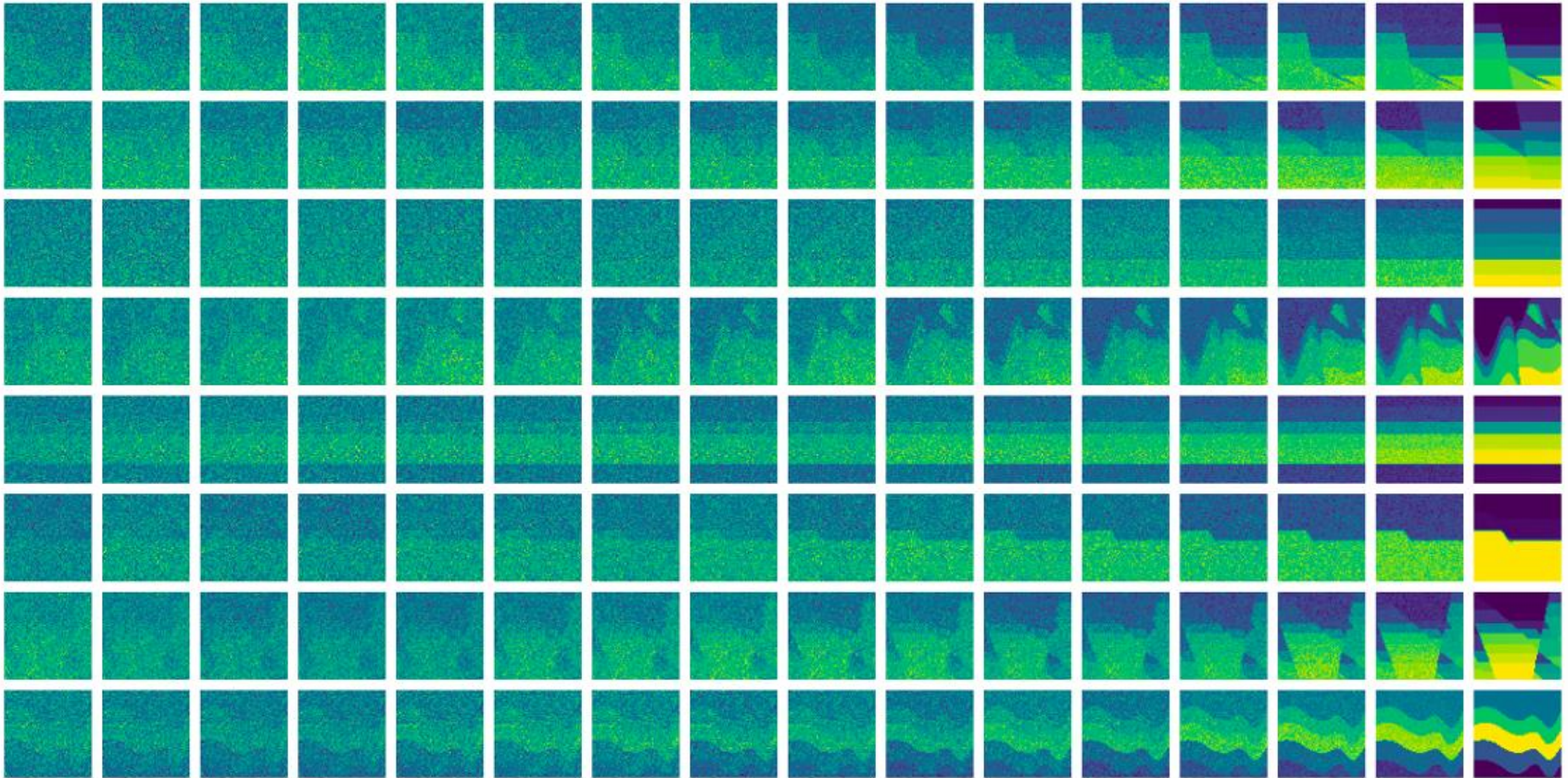
Mapping distributions



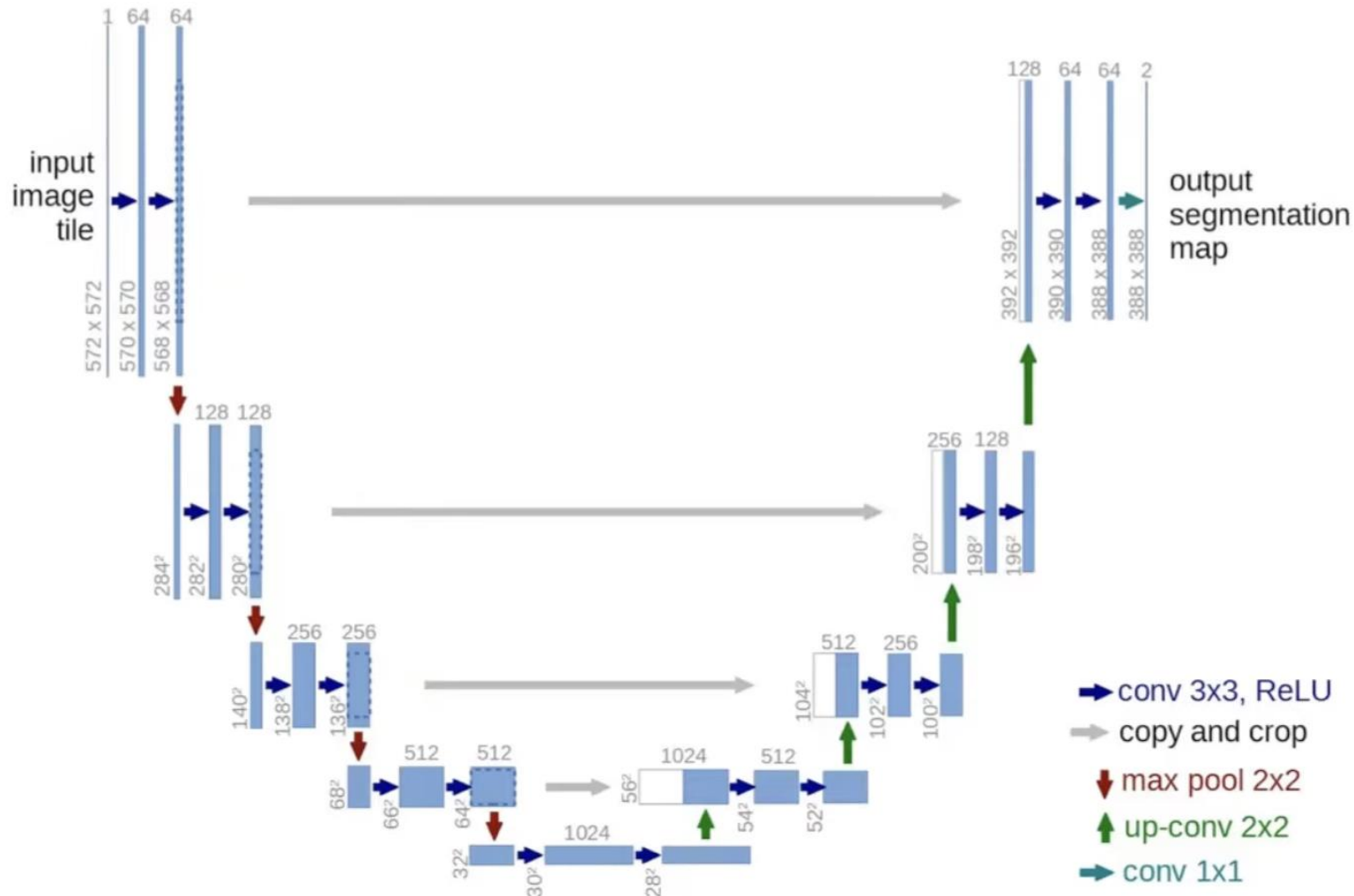
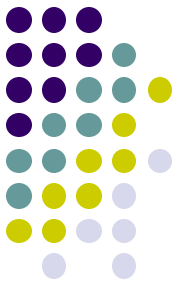
The reverse diffusion progress



Here shows the velocity generation progress every 8 time steps in the final 120 timesteps

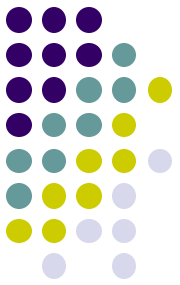


A Unet

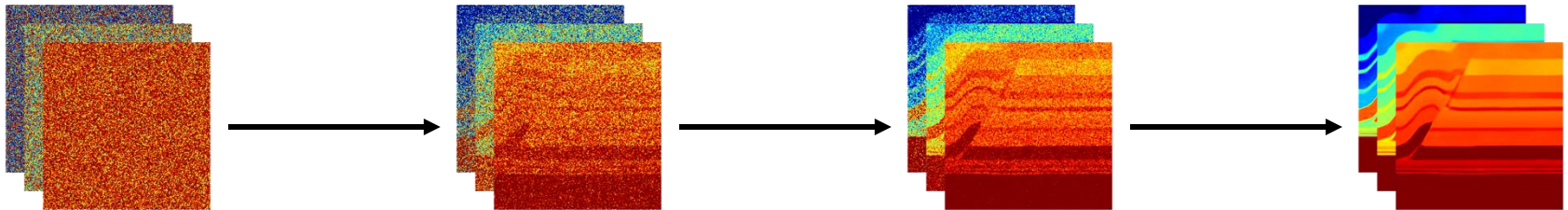


Same size input and output

Elastic velocity model

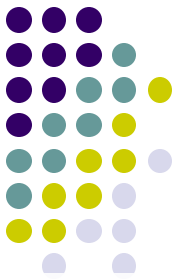


$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \mathbf{z}_{t-1}$$

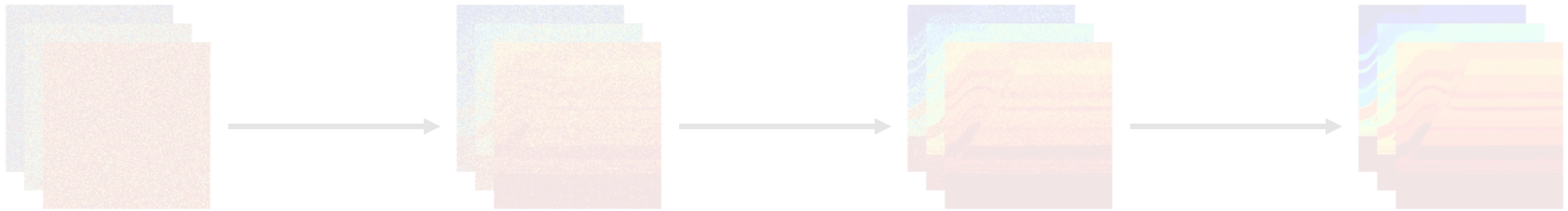


$$\mathbf{x}_{i-1} = \frac{1}{\sqrt{1 - \beta_i}} (\mathbf{x}_i + \beta_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, i)) + \sqrt{\beta_i} \mathbf{z}_i$$

Classifier-free guidance



$$\mathbf{x}_i = \sqrt{1 - \beta_i} \mathbf{x}_{i-1} + \sqrt{\beta_i} \mathbf{z}_{i-1}$$



Unconditional

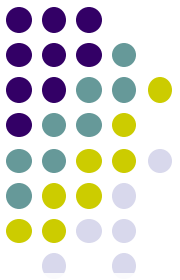
$$\mathbf{x}_{i-1} = \frac{1}{\sqrt{1 - \beta_i}} (\mathbf{x}_i + \beta_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, i)) + \sqrt{\beta_i} \mathbf{z}_i$$

Conditional

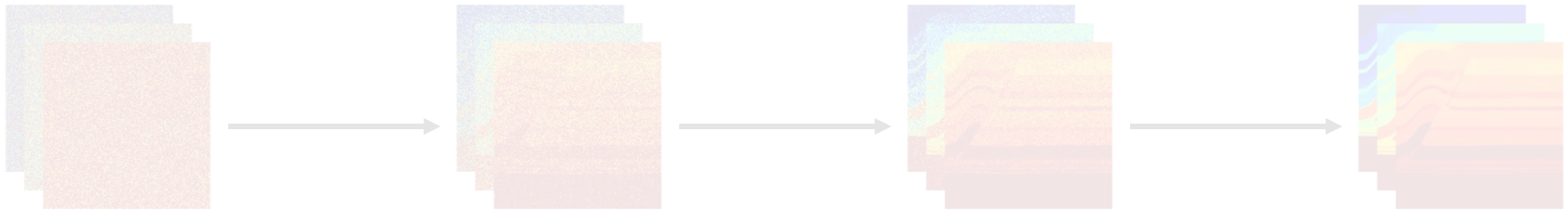
$$\mathbf{x}_{i-1} = \frac{1}{\sqrt{1 - \beta_i}} (\mathbf{x}_i + \beta_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, i, C)) + \sqrt{\beta_i} \mathbf{z}_i$$

Uncertainty? and Generalization?

Reconstruction guidance



$$\mathbf{x}_i = \sqrt{1 - \beta_i} \mathbf{x}_{i-1} + \sqrt{\beta_i} \mathbf{z}_{i-1}$$



Unconditional

$$\mathbf{x}_{i-1} = \frac{1}{\sqrt{1-\beta_i}} (\mathbf{x}_i + \beta_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, i)) + \sqrt{\beta_i} \mathbf{z}_i$$

Conditional

$$\mathbf{x}_{i-1} = \underbrace{\frac{1}{\sqrt{1-\beta_i}} (\mathbf{x}_i + \beta_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, i))}_{\text{Diffusion}} + \underbrace{\sqrt{\beta_i} \mathbf{z}_i}_{\text{Noise}} + \underbrace{\nabla_{\mathbf{x}_i} (\mathcal{F}(\mathbf{x}_i), \mathcal{D})}_{\text{Condition Gradient}}$$

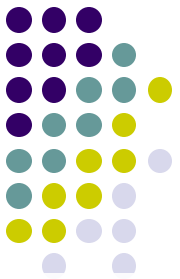
Diffusion

Noise

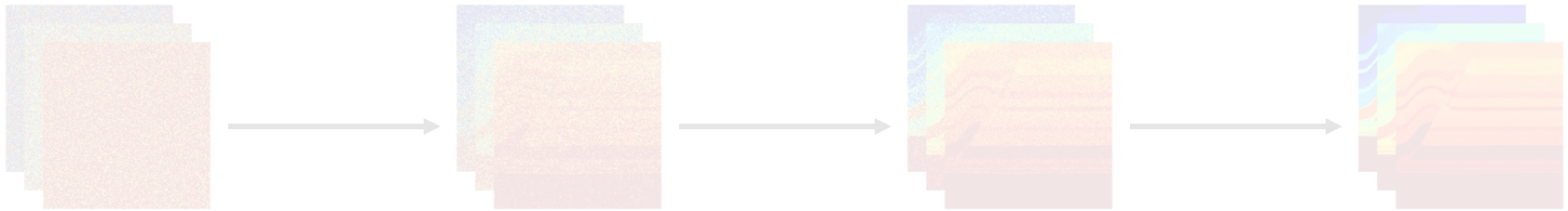
Condition Gradient

Generalization?

Reconstruction guidance



$$\mathbf{x}_i = \sqrt{1 - \beta_i} \mathbf{x}_{i-1} + \sqrt{\beta_i} \mathbf{z}_{i-1}$$



Unconditional

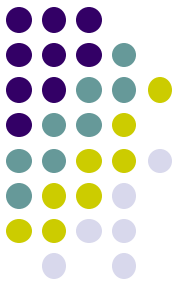
$$\mathbf{x}_{i-1} = \frac{1}{\sqrt{1 - \beta_i}} (\mathbf{x}_i + \beta_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, i)) + \sqrt{\beta_i} \mathbf{z}_i$$

Conditional

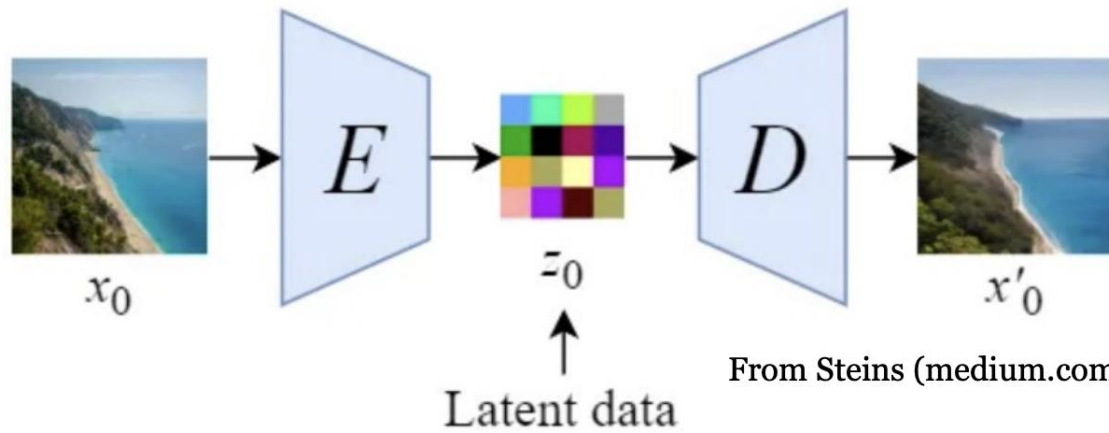
$$\mathbf{x}_{i-1} = \frac{1}{\sqrt{1 - \beta_i}} (\mathbf{x}_i + \beta_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, i)) + \sqrt{\beta_i} \mathbf{z}_i + \nabla_{\mathbf{x}_i} (\mathcal{F}(\mathbf{x}_i), \mathcal{D})$$

- ✓ How many gradient steps should be used?
- ✓ How does it compare with the cost of unconditional sampling?

Latent Diffusion Models (Stable Diffusion)

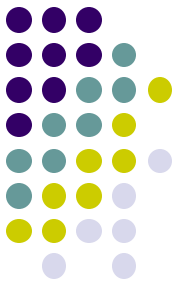


- Rombach, Blattman et al., 2022
- **Speed up:** performing the diffusion in a low dimensional latent space
- **Conditional generation:** condition denoising on text, images, etc.



From Steins (medium.com)

Latent diffusion



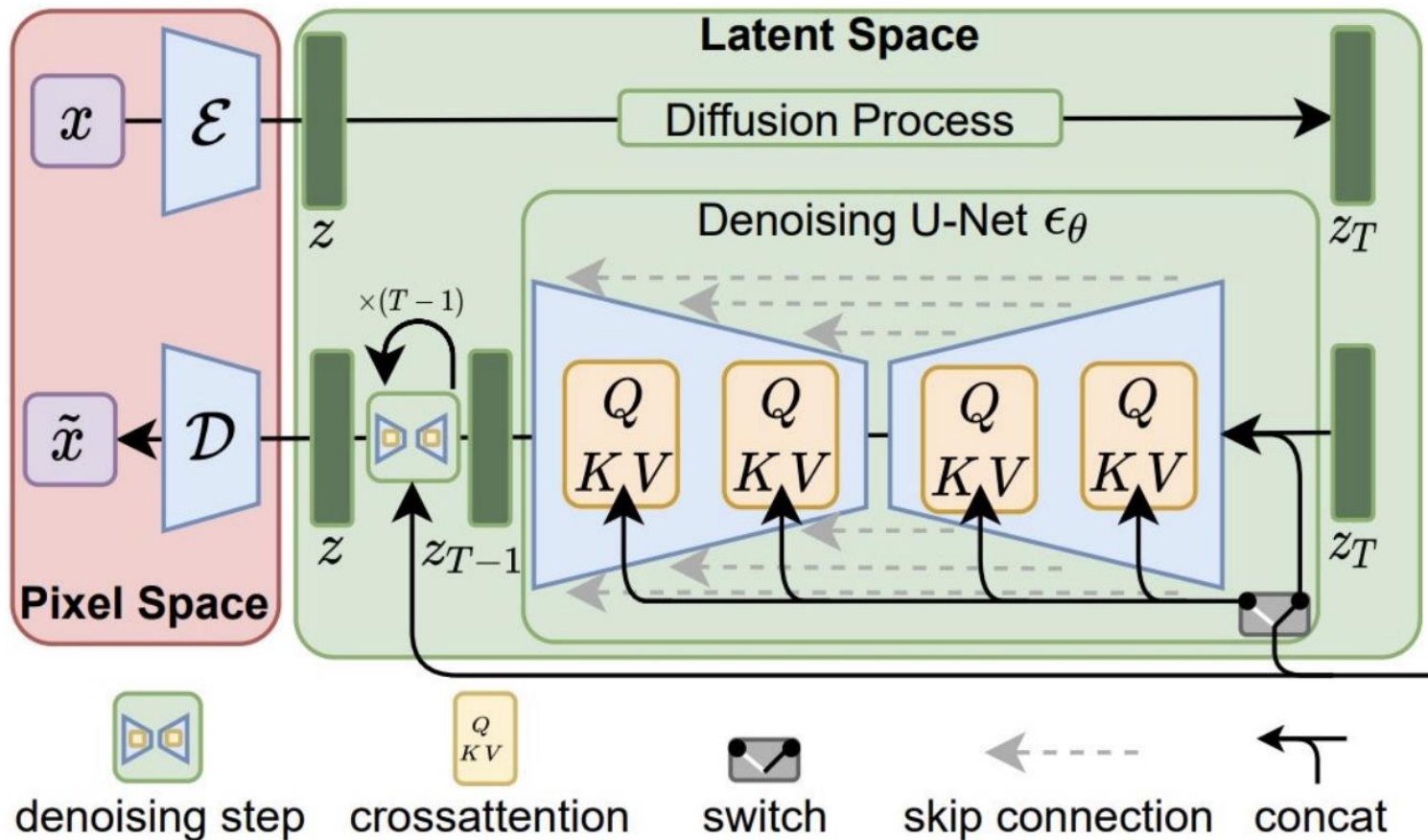
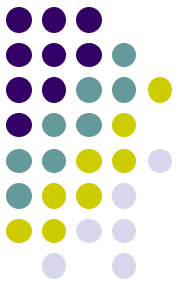
$$\bullet + \Sigma = \bullet$$

Pixel Noise Pixel

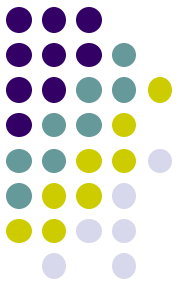
$$\bullet + \Sigma = \bullet$$

Latent Noise Latent

Latent diffusion



Diffusion FWI



The FWI objective function with regularization term:

$$J(\mathbf{m}) = \frac{1}{2} \|\mathbf{d}_{\text{obs}} - \mathbf{F}(\mathbf{m})\|_2^2 + \mathbf{R}(\mathbf{m})$$

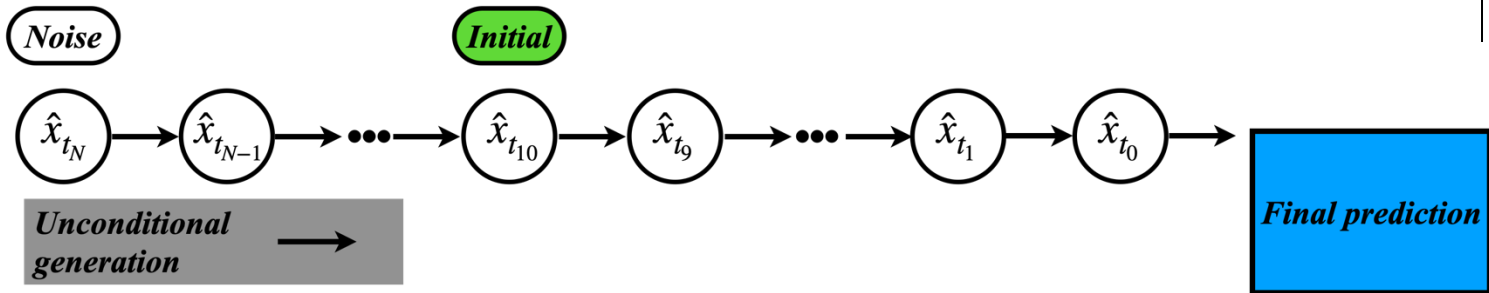
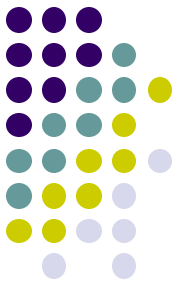
Its solution depends on the proximal operator of the regularization:

$$\mathbf{m}_{k+1} = \text{prox}_{\mathbf{R}} \left(\mathbf{m}_k + \alpha_k \frac{\partial J}{\partial \mathbf{m}_k} \right)$$

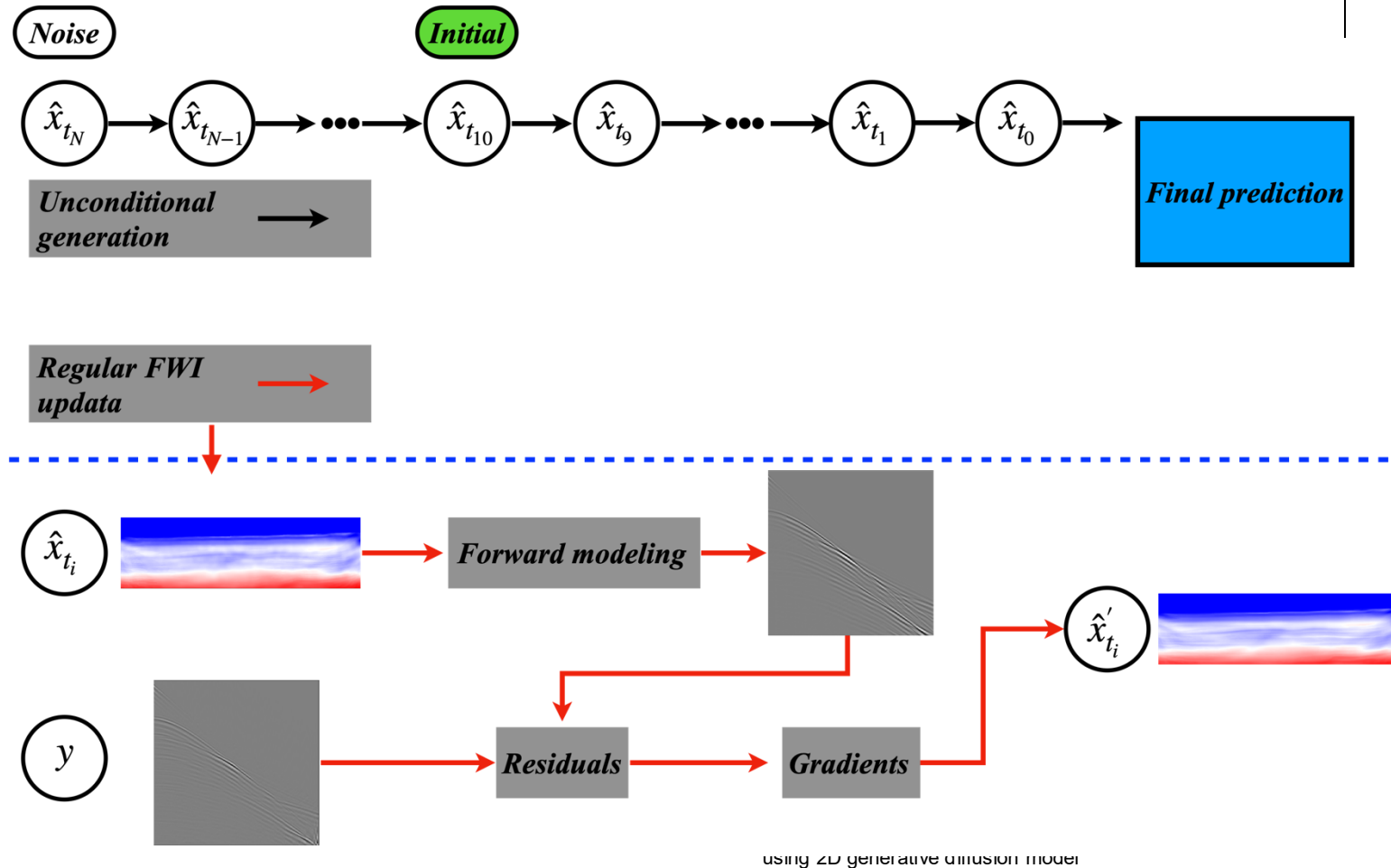
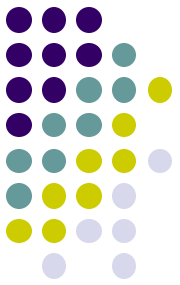
Replace it by reverse Diffusion progress:

$$\mathbf{m}_{k+1} = \mathbf{f} \left(\mathbf{m}_k + \alpha_k \frac{\partial J}{\partial \mathbf{m}_k}, \mathbf{s}_{\theta^*}, t \right) + \sqrt{\beta(t)} d\mathbf{w}$$

Diffusion FWI framework

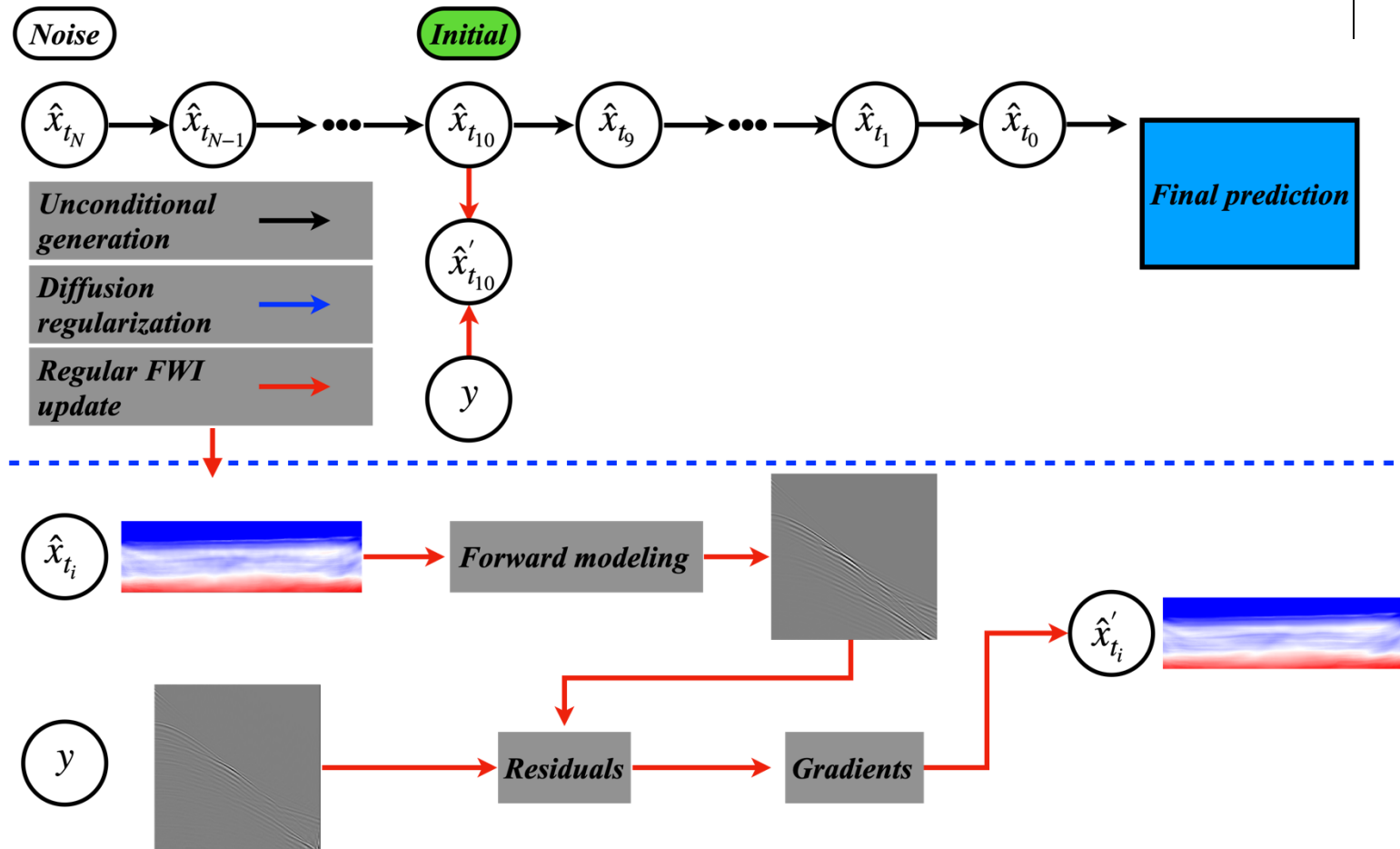
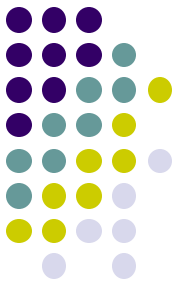


Diffusion FWI framework



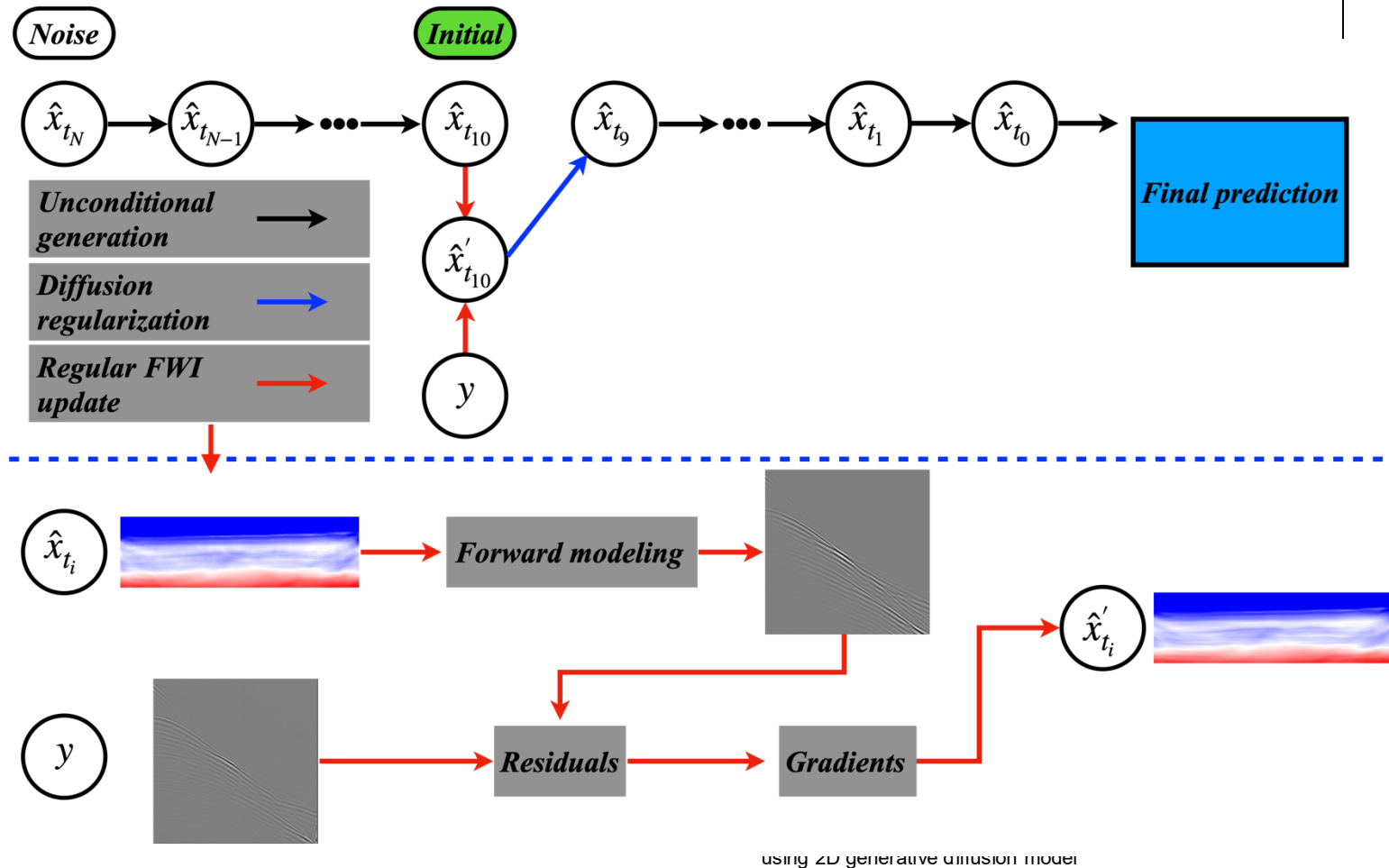
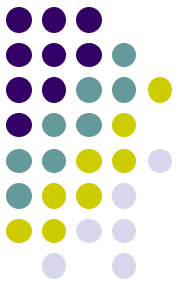
using 2D generative diffusion model

Diffusion FWI framework



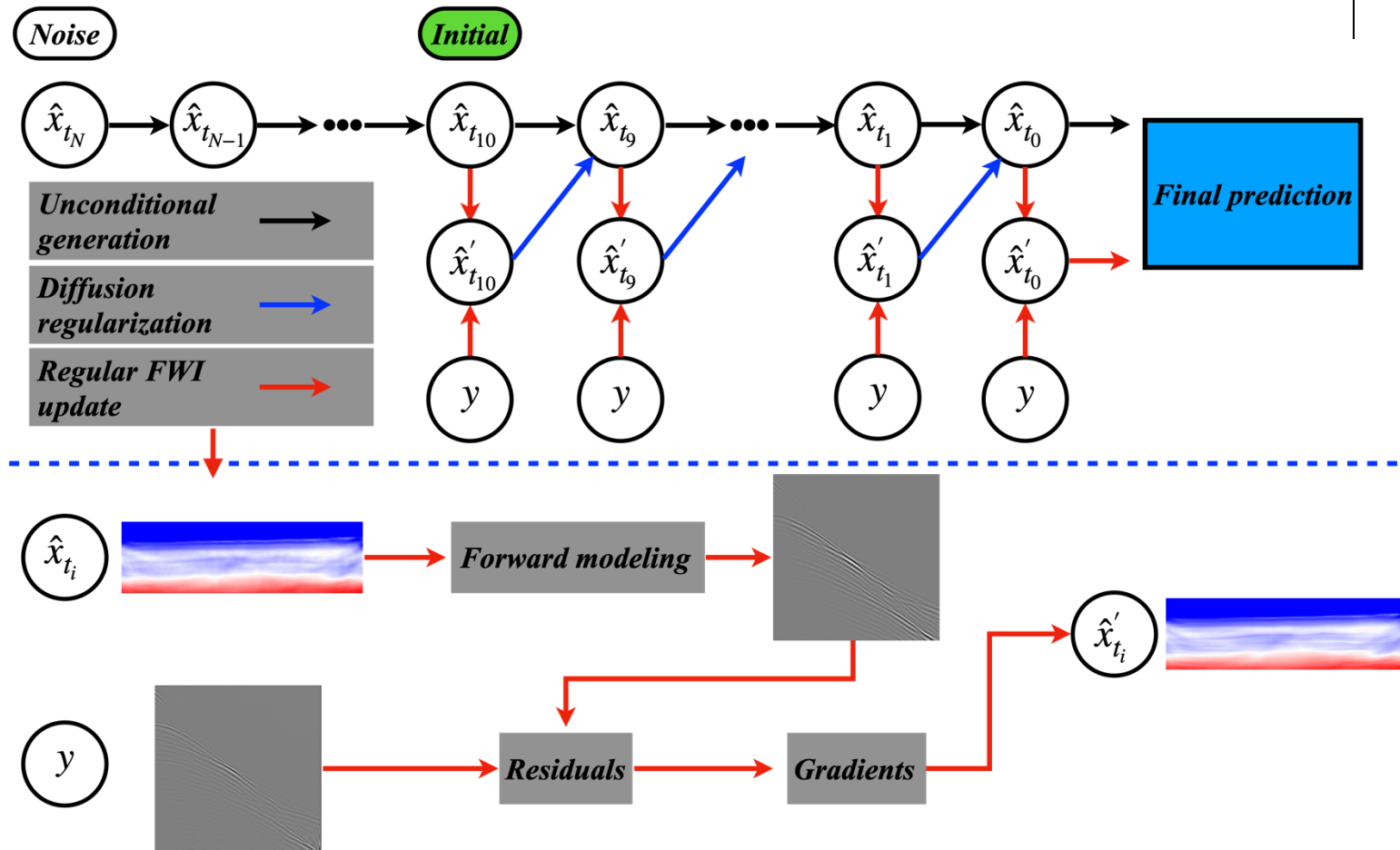
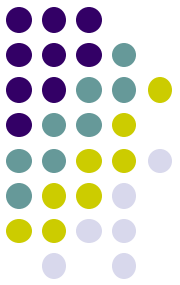
using 2D generative diffusion model

Diffusion FWI framework



using 2D generative diffusion model

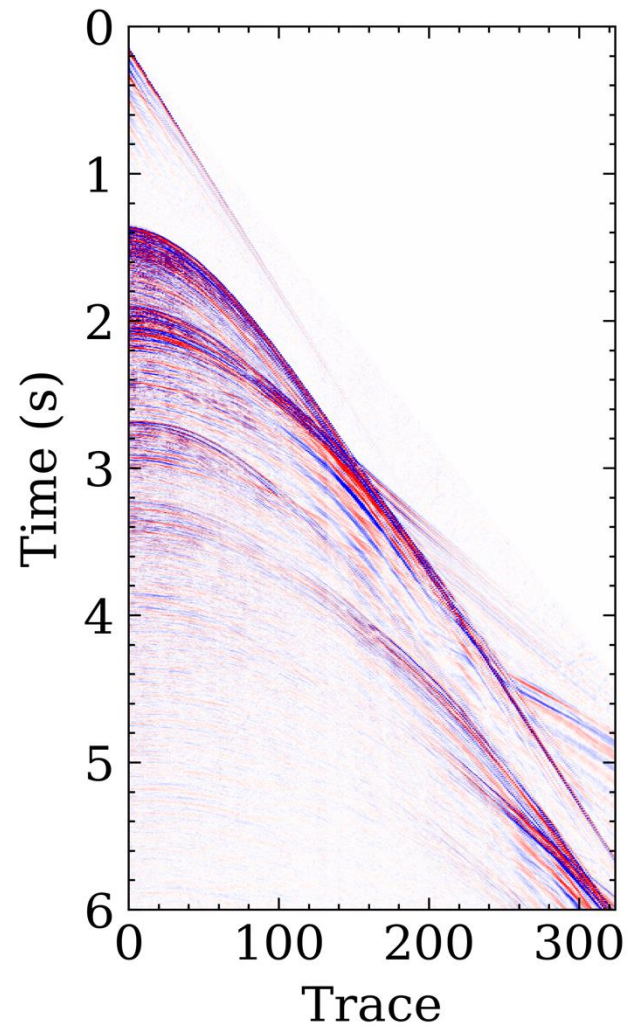
Diffusion FWI framework



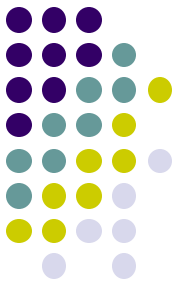
using 2D generative diffusion model

Marine Field

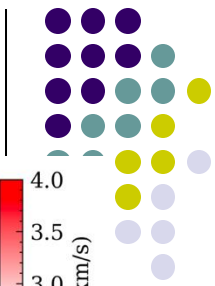
Total 116 shots



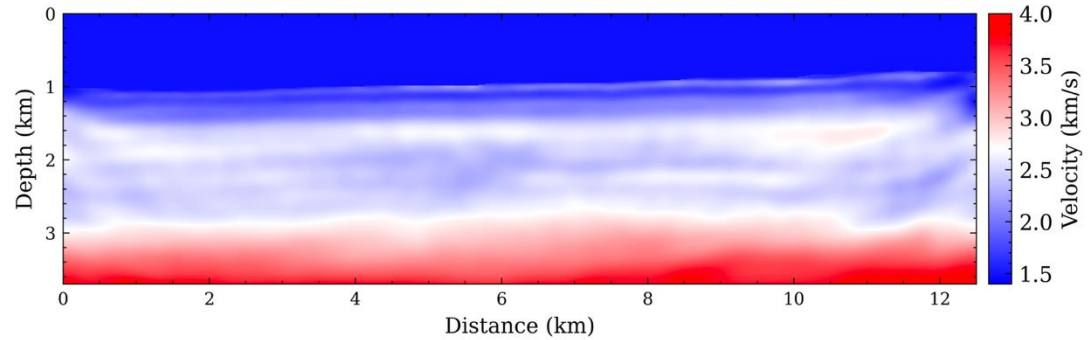
using 2D generative diffusion model



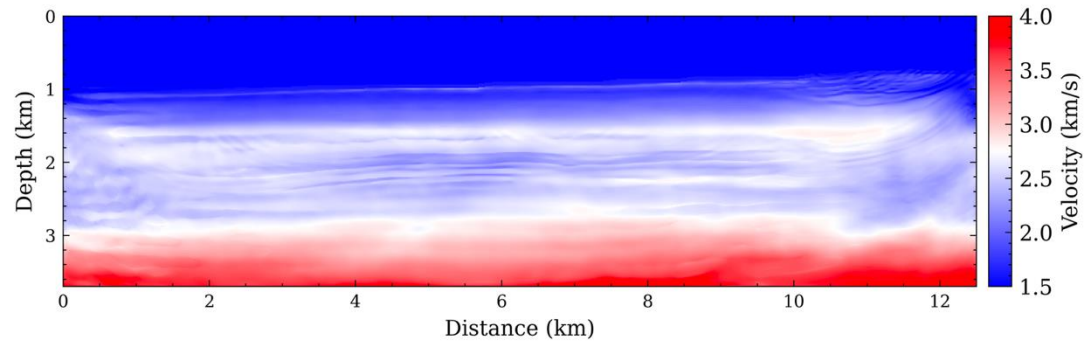
Field Data results



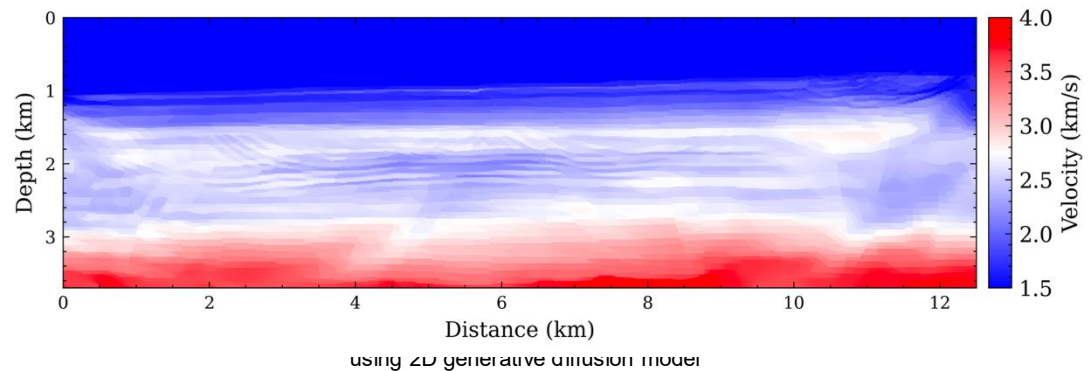
Initial velocity model



Conventional FWI

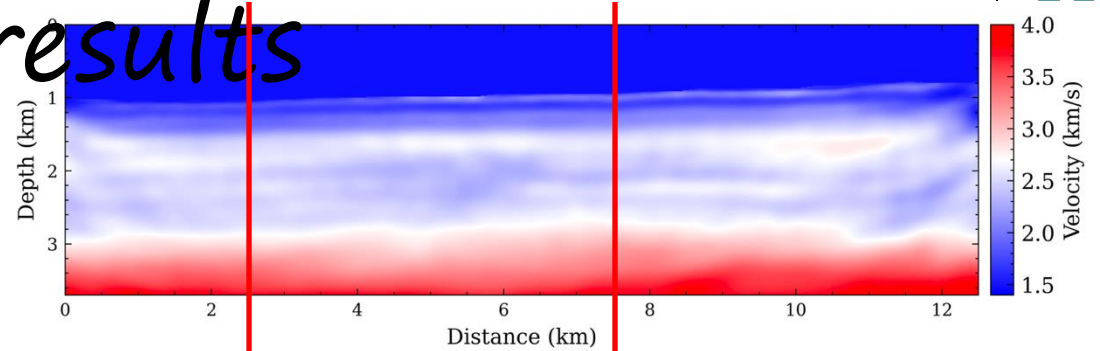


Diffusion FWI by
OpenFWI dataset

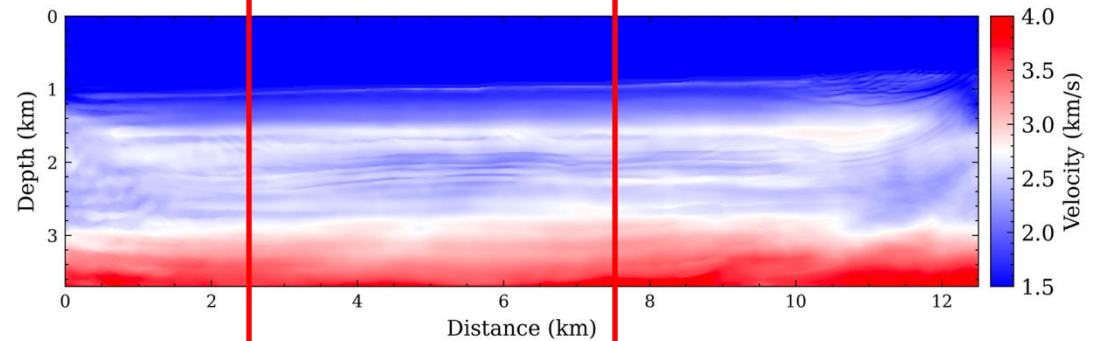


Field Data results

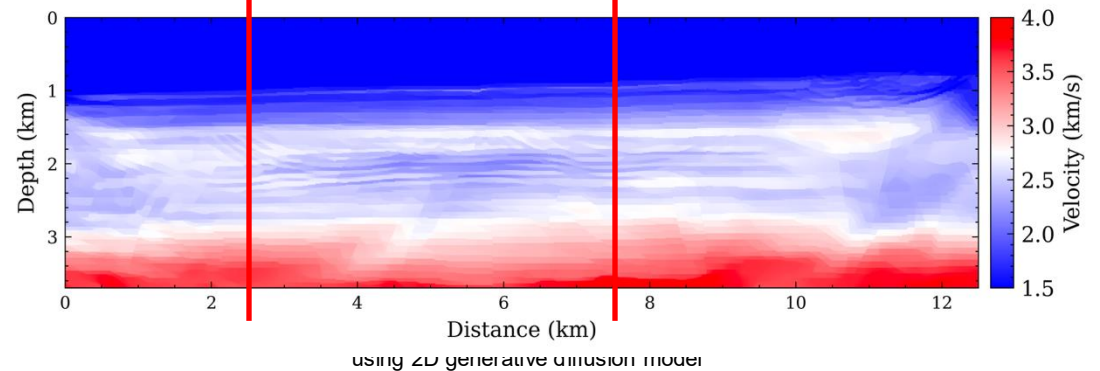
Initial velocity model



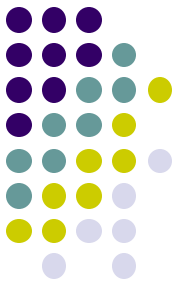
Conventional FWI



Diffusion FWI by
OpenFWI dataset

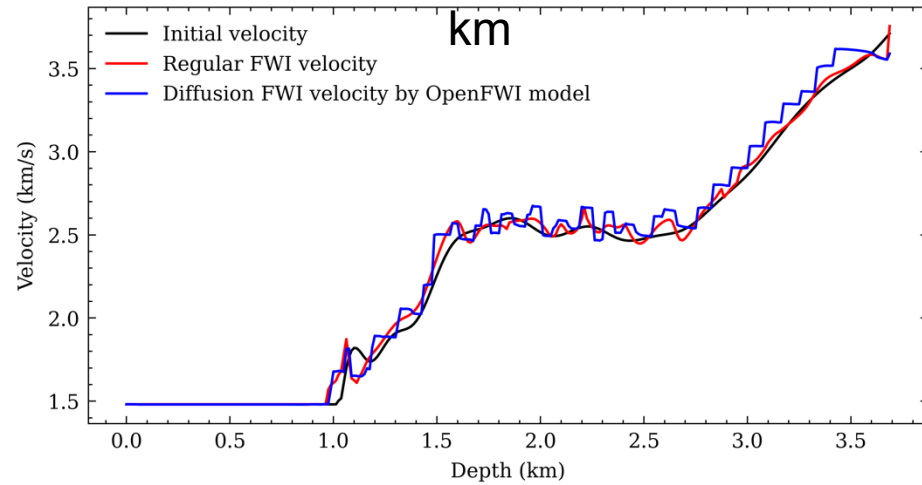


Profile comparisons



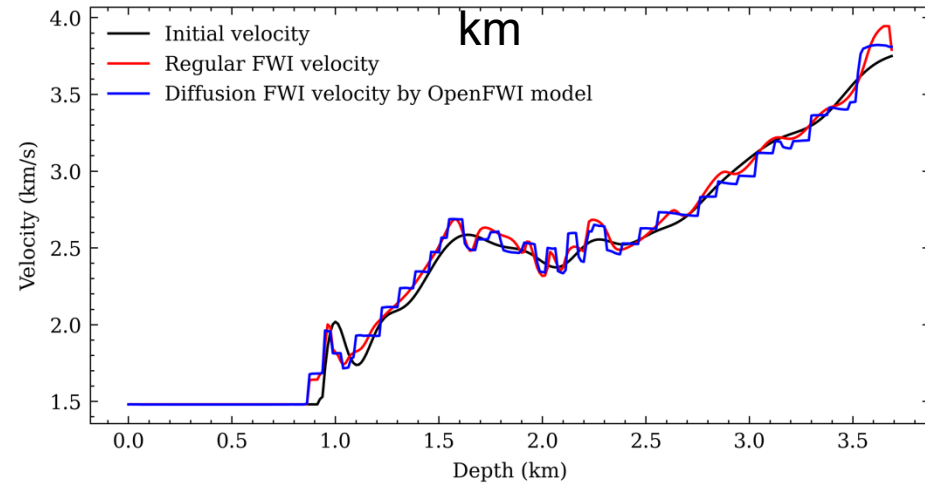
2.5

km



7.5

km

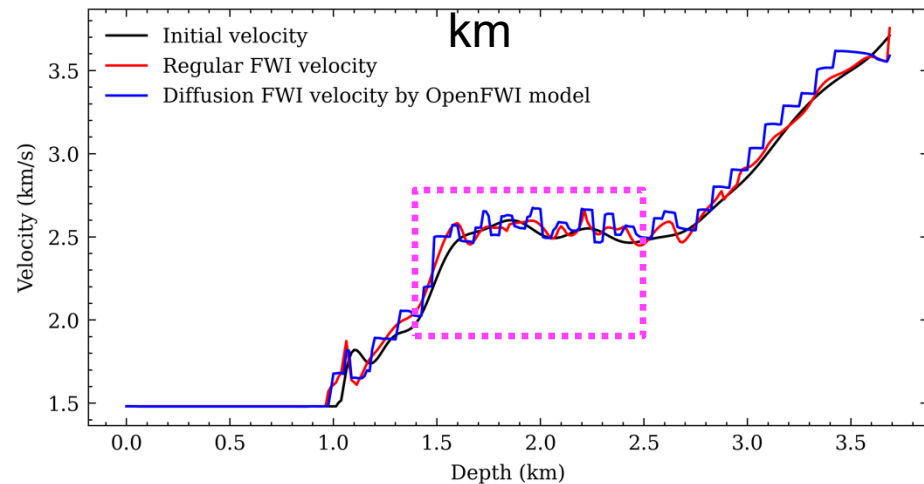


Profile comparisons



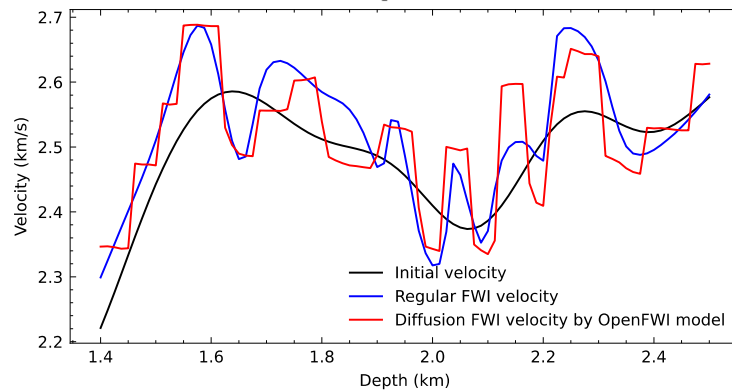
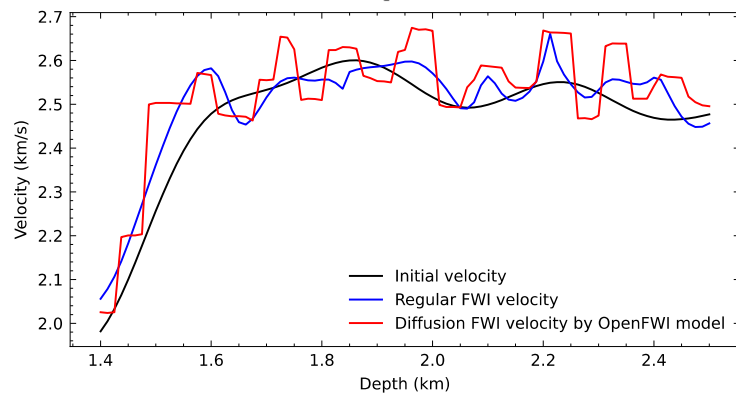
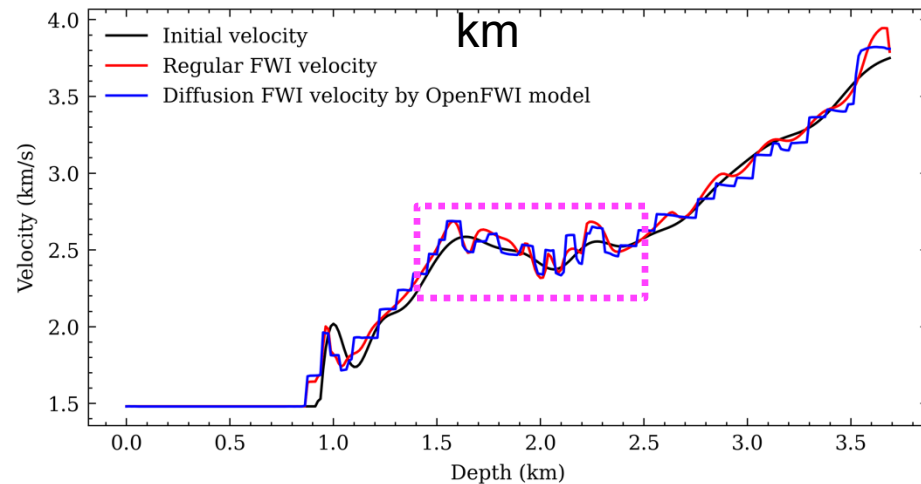
2.5

km

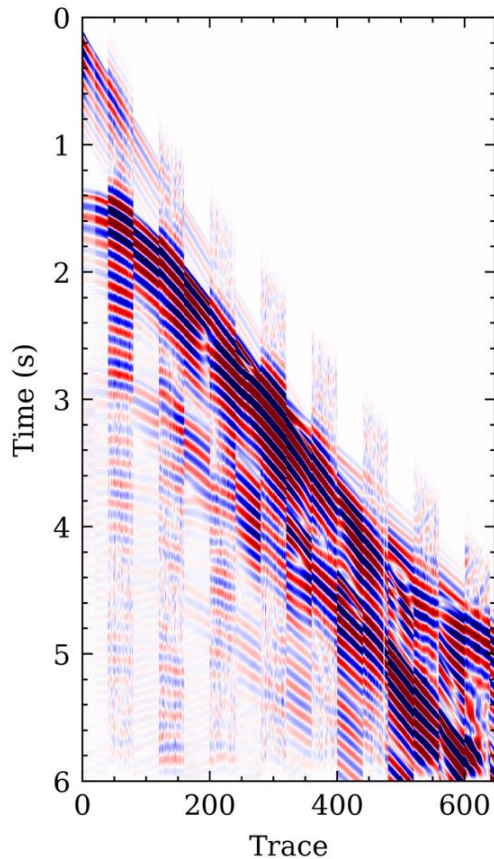
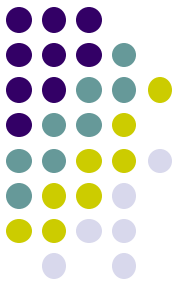


7.5

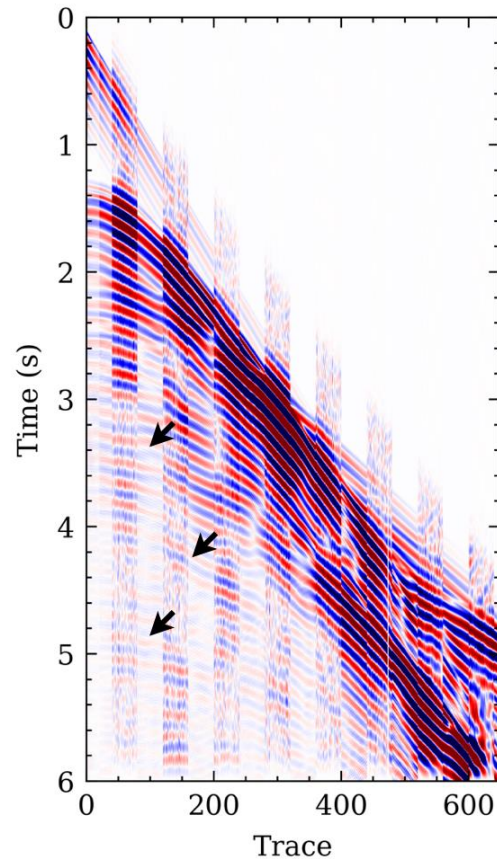
km



The interleaved predicted and observed data

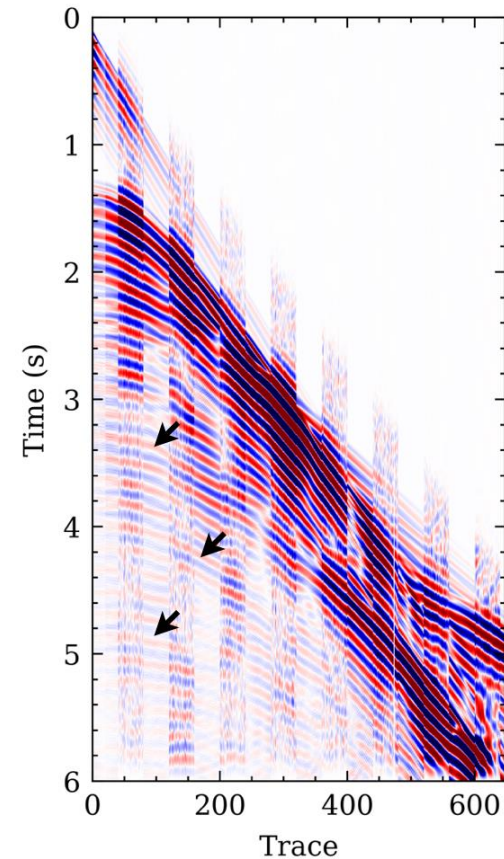


The initial
velocity



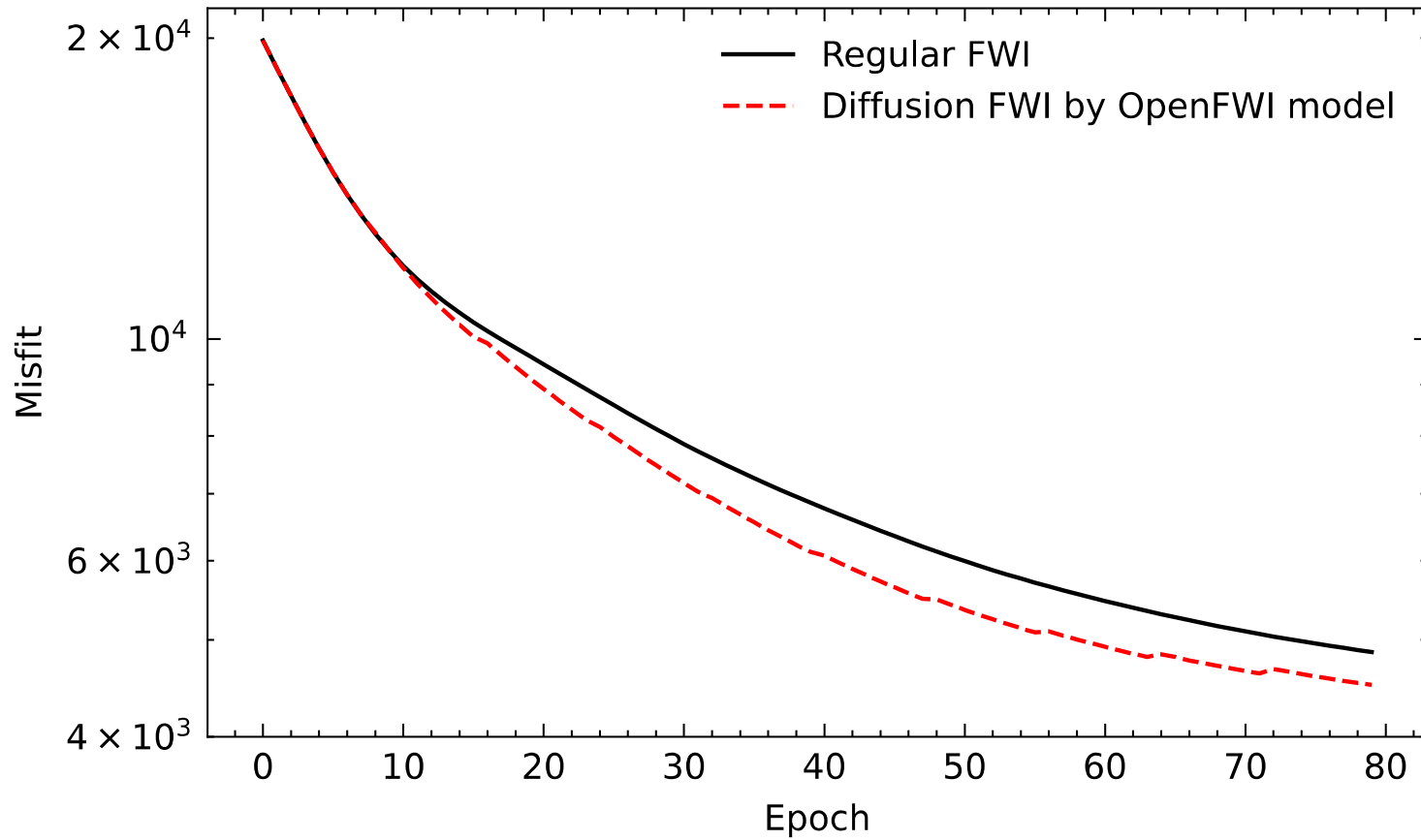
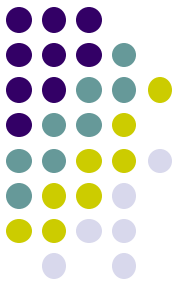
Conventional
FWI

using 2D generative diffusion model

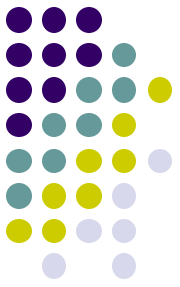


Diffusion FWI

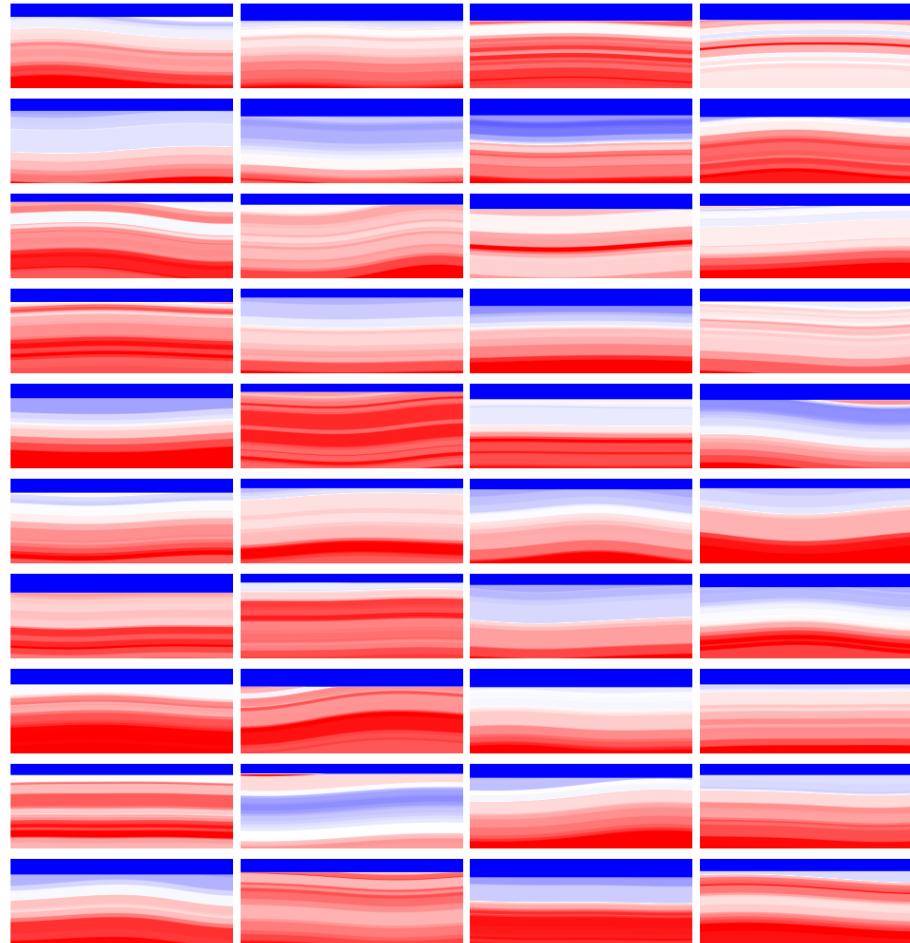
Data misfit



Random model based on well data



Velocity model size:
(296,1000)

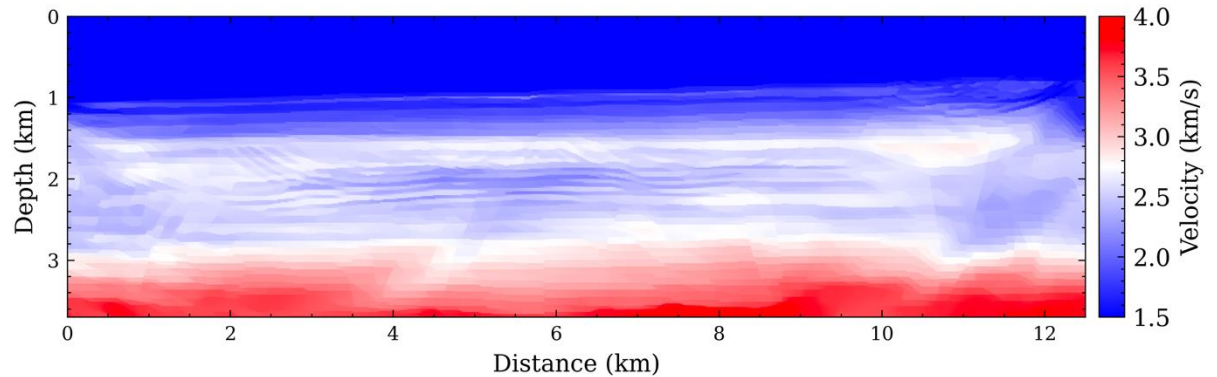


using 2D generative diffusion model

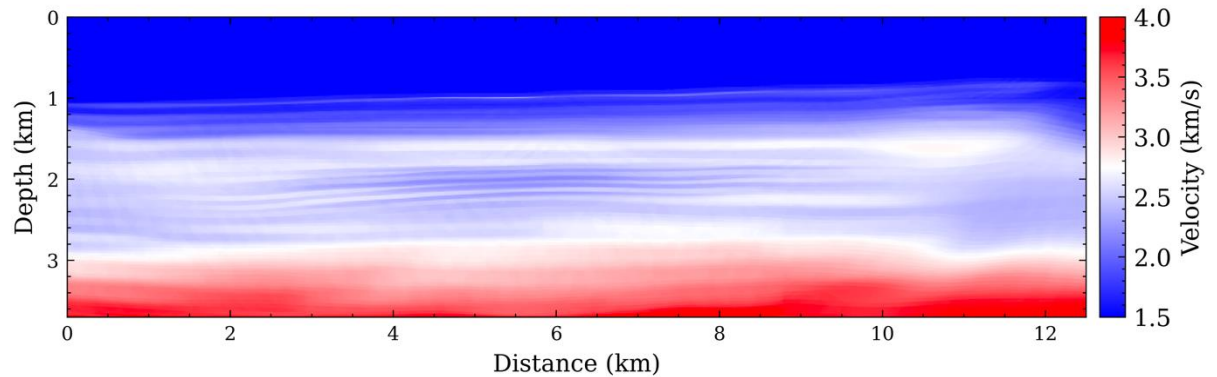
Diffusion FWI results



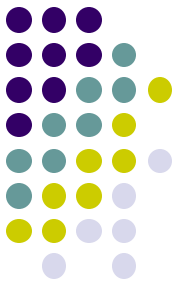
Diffusion FWI by
OpenFWI dataset



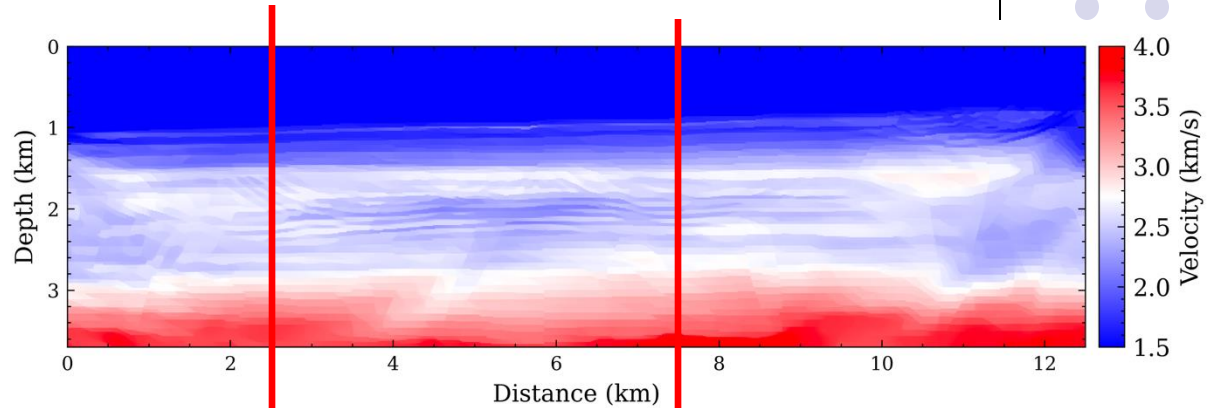
Diffusion FWI by
random model



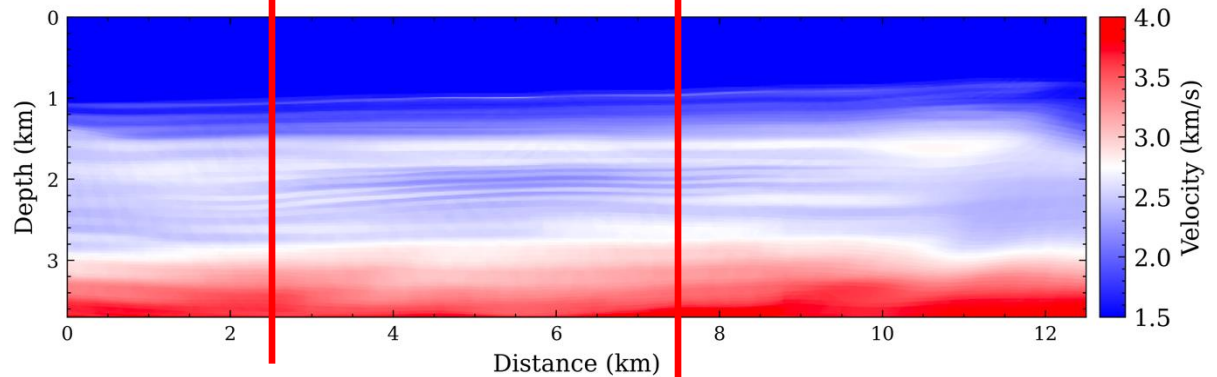
Diffusion FWI results



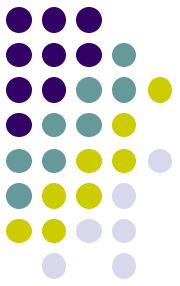
Diffusion FWI by
OpenFWI dataset



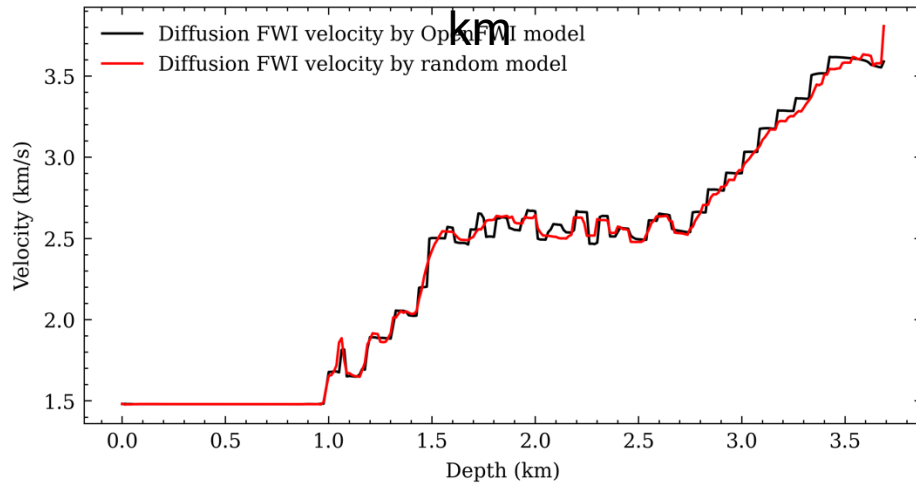
Diffusion FWI by
random model



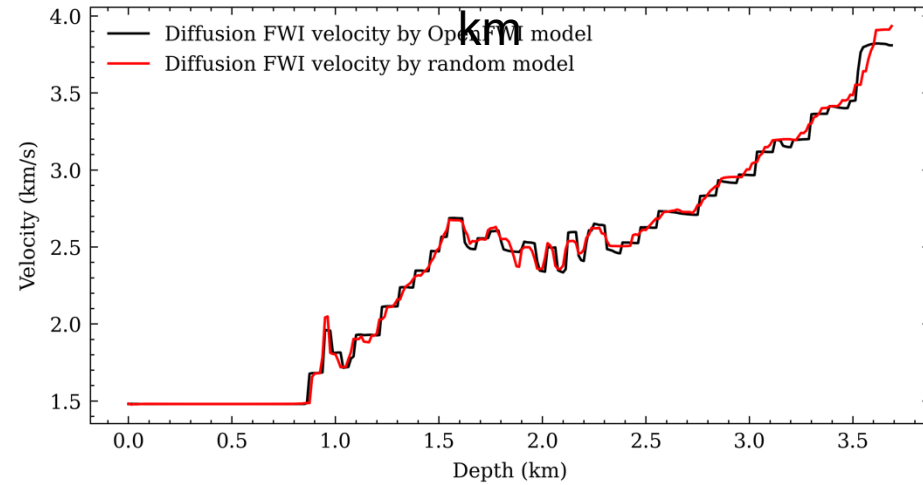
Profile comparisons



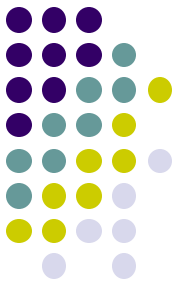
2.5



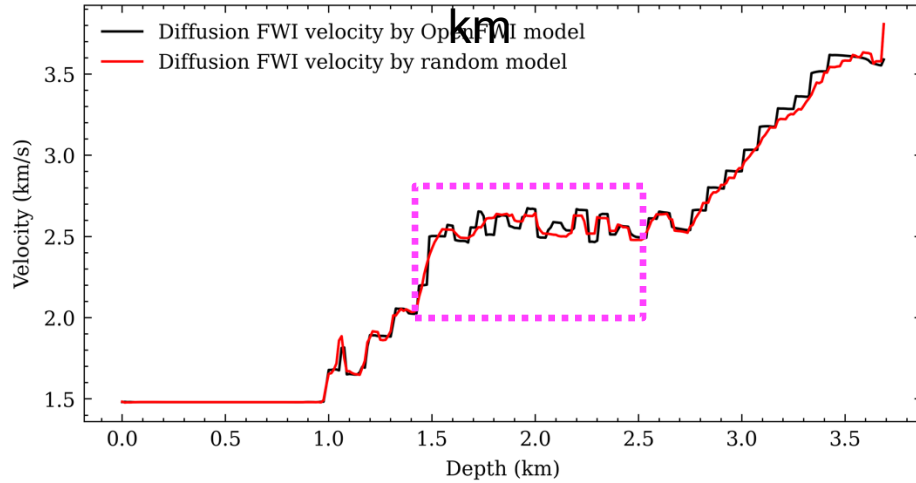
7.5



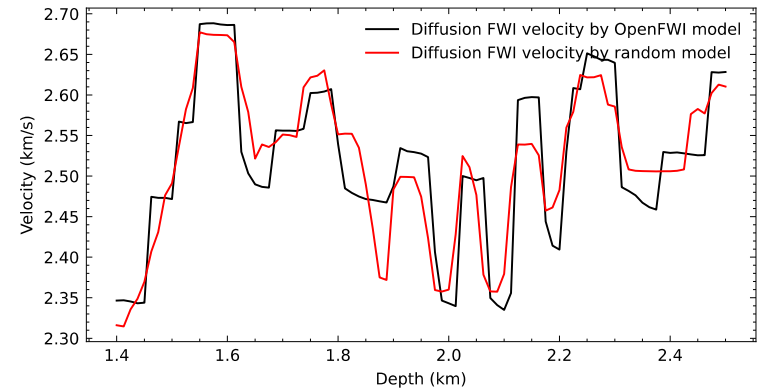
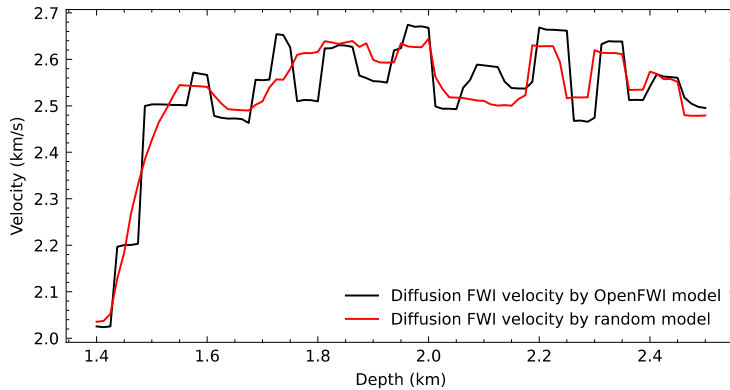
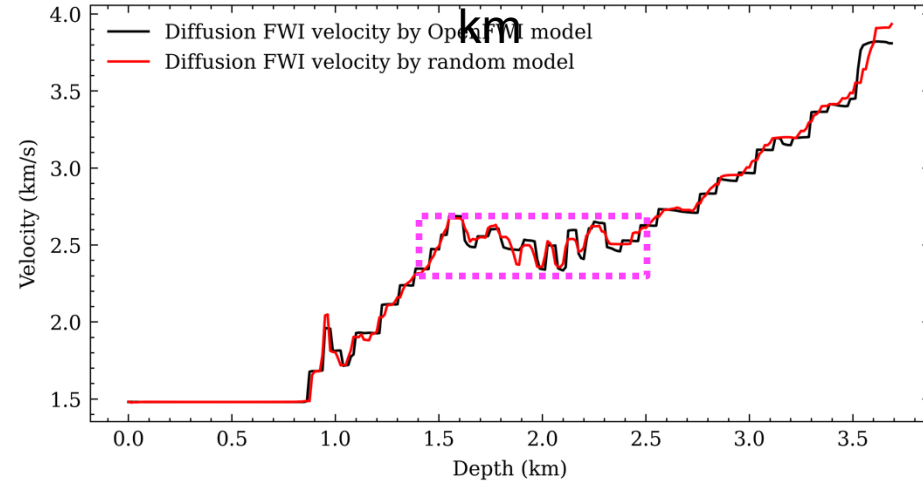
Profile comparisons



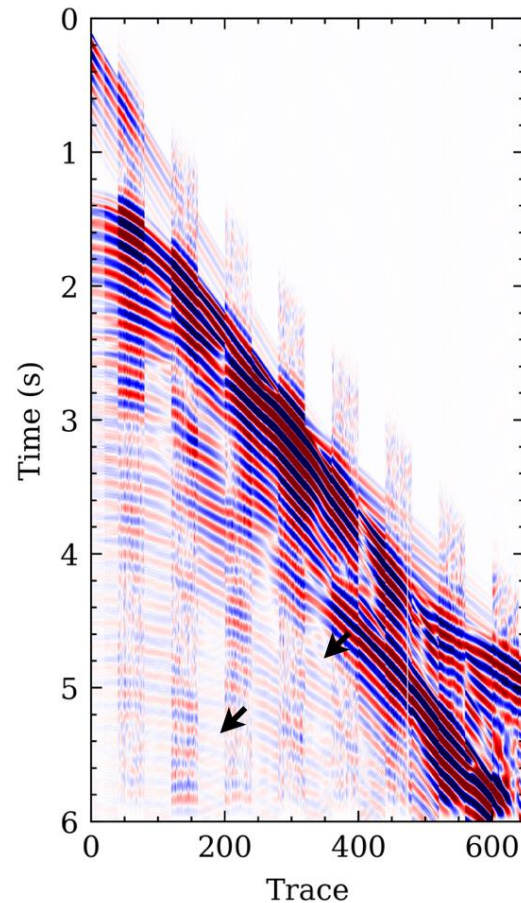
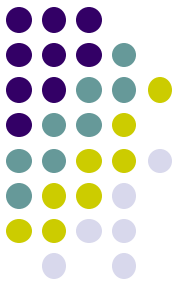
2.5
km



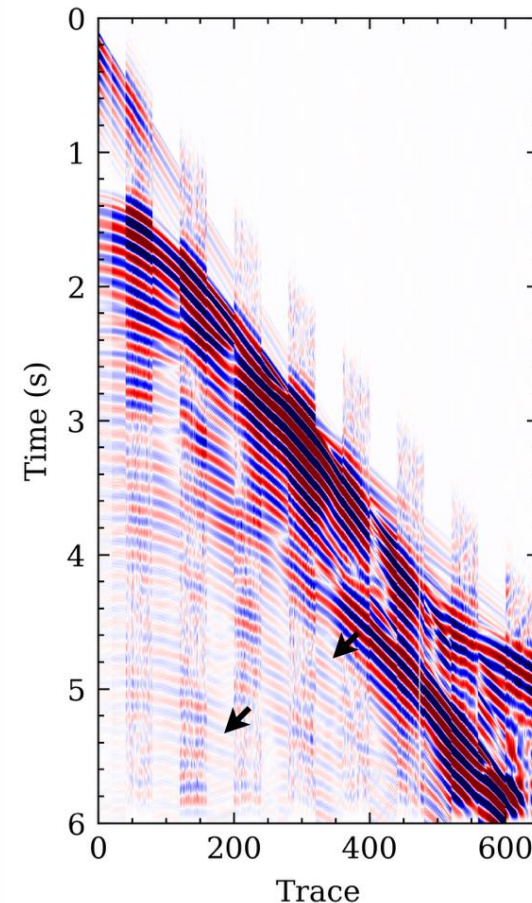
7.5
km



The interleaved predicted and observed data

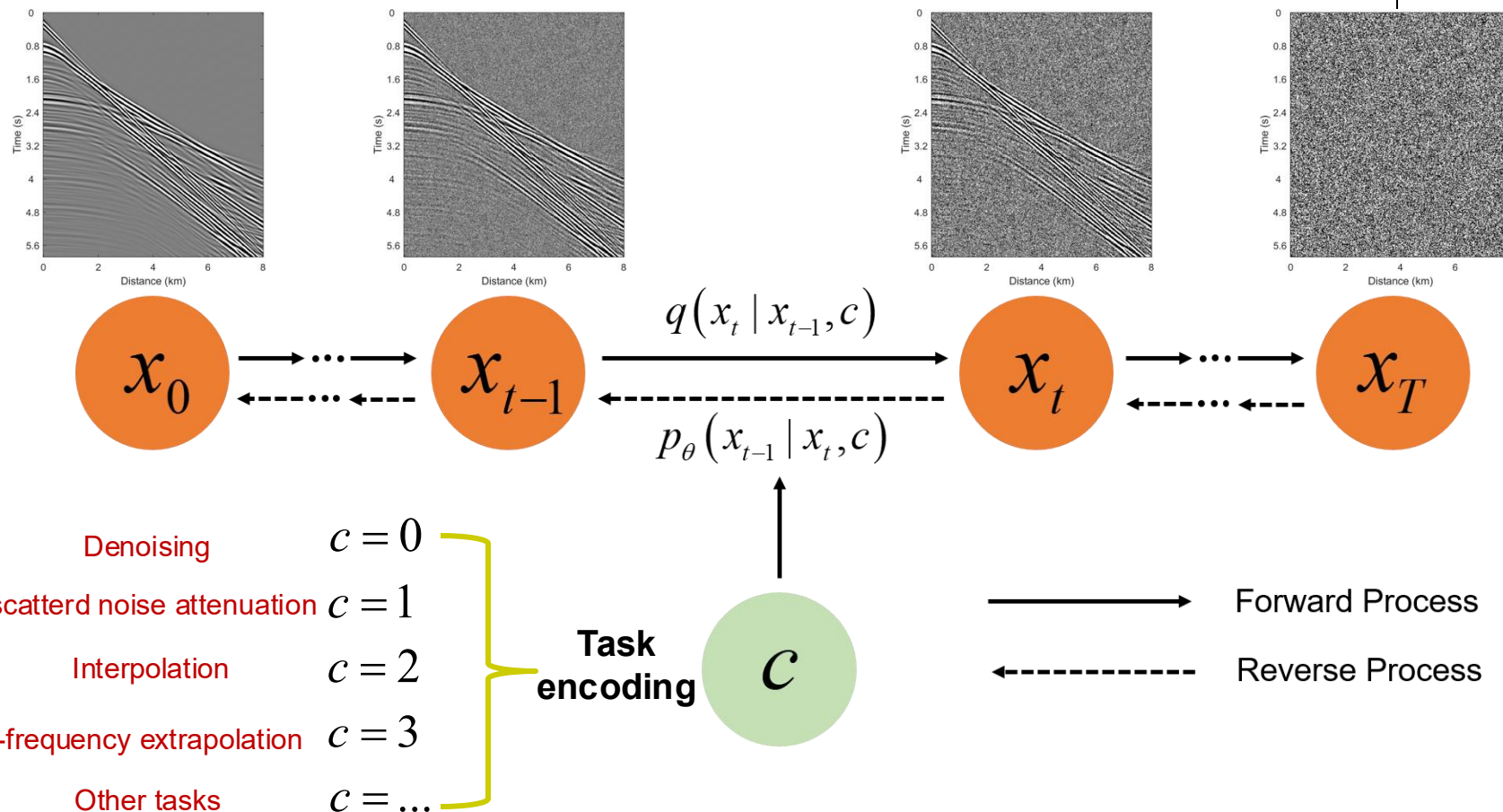


Diffusion FWI by OpenFWI model

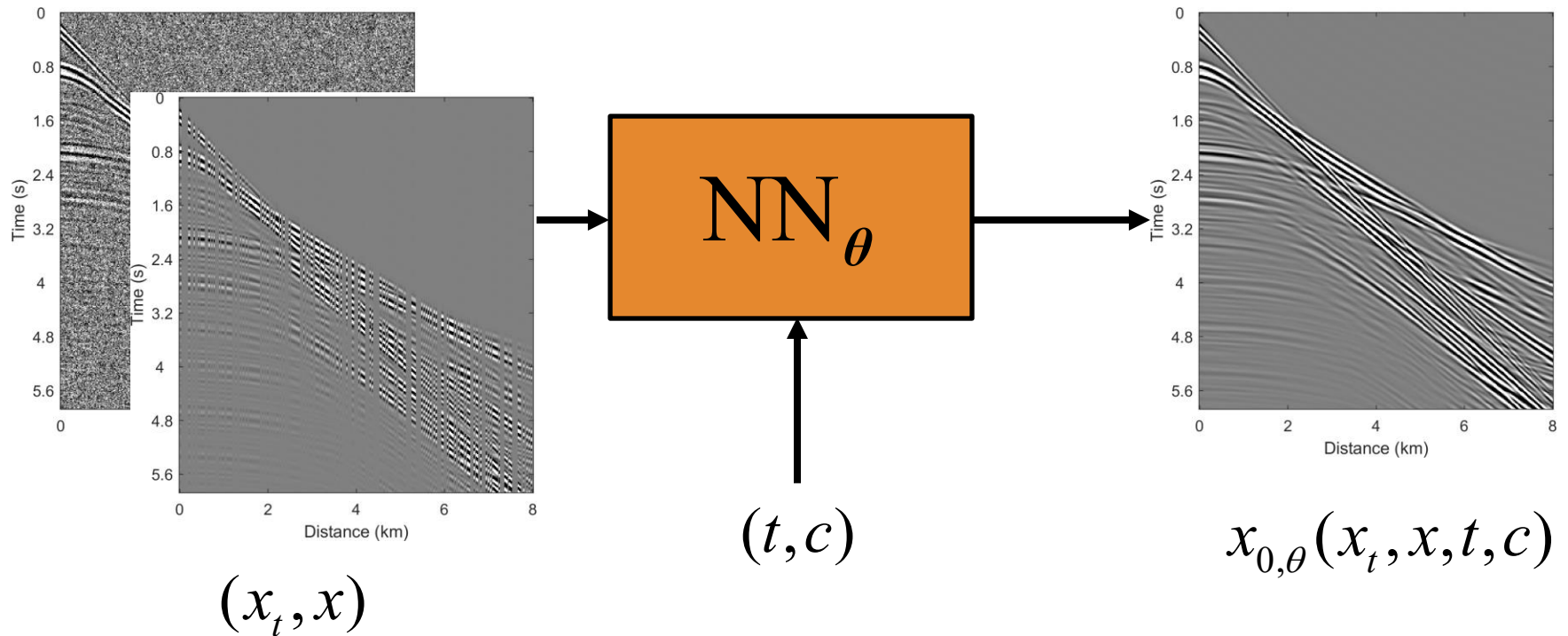
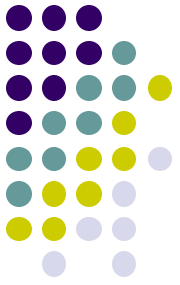


Diffusion FWI by random model
using 2D generative diffusion model

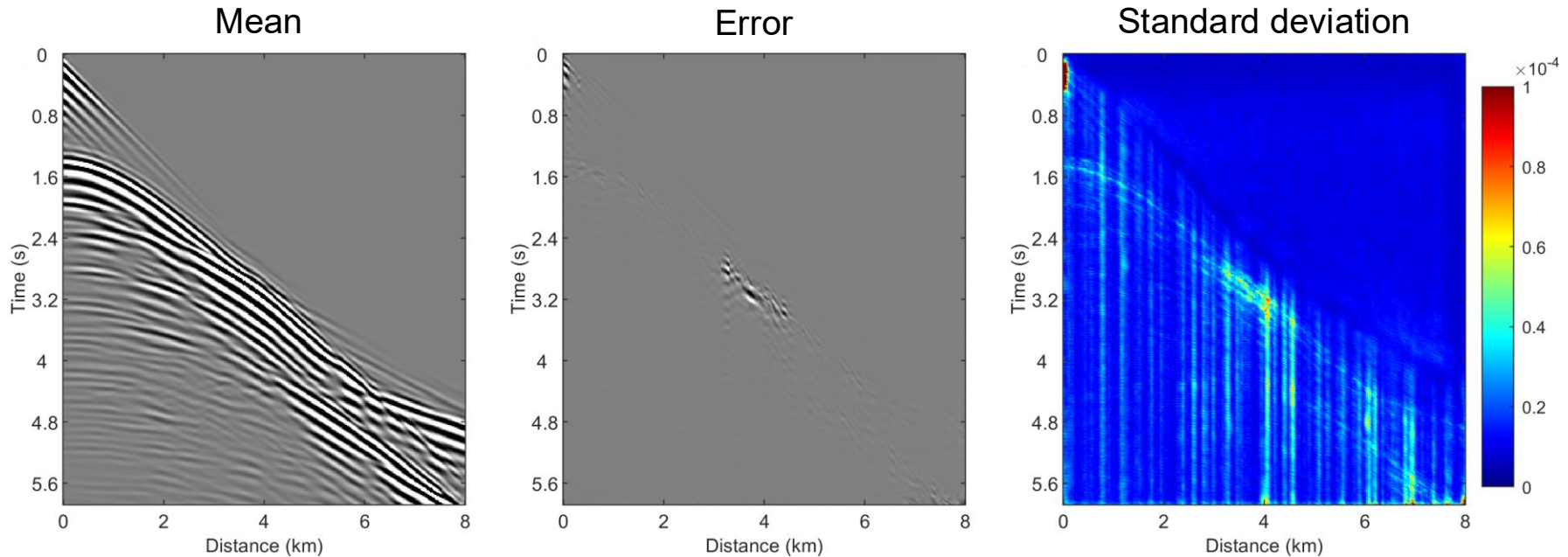
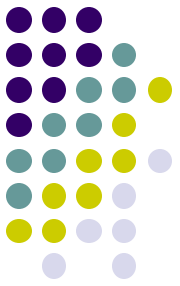
Illustration of our GSFM



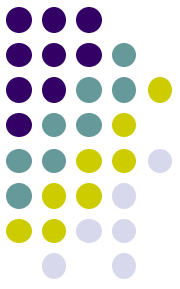
Pretraining on synthetic data: Interpolation



Uncertainty quantification using pretrained GSFM



Conclusions



- The generative power has only begun, stay tuned
- Sampling from complex distributions in inverse problems
- We can incorporate our expectations, the data and physics
- Applications include tomography, prediction, inversion,.....